

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

Trịnh Huy Hoàng

THIẾT KẾ HỆ THỐNG SOC DỰA TRÊN
CPU RISC-V 64-BIT ĐA LỖI ĐỒNG NHẤT VÀ
NETWORK-ON-CHIP TÍCH HỢP
CÁC LỖI MẬT MÃ HÓA NHẸ TRÊN FPGA

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Tp. Hồ Chí Minh - Năm 2025

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

THIẾT KẾ HỆ THỐNG SOC DỰA TRÊN
CPU RISC-V 64-BIT ĐA LỖI ĐỒNG NHẤT VÀ
NETWORK-ON-CHIP TÍCH HỢP
CÁC LỖI MẬT MÃ HÓA NHẹ TRÊN FPGA

Họ và tên sinh viên: Trịnh Huy Hoàng
Mã số SV: 21207156

NGƯỜI HƯỚNG DẪN KHOA HỌC
PGS.TS. Lê Đức Hùng

Tp. Hồ Chí Minh - Năm 2025

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến PGS.TS. Lê Đức Hùng - người thầy đã tận tâm hướng dẫn và đồng hành cùng em trong suốt quá trình thực hiện khóa luận tốt nghiệp. Thầy không chỉ truyền đạt kiến thức chuyên môn quý báu mà còn tạo nhiều điều kiện thuận lợi để em phát triển ý tưởng nghiên cứu và triển khai đề tài một cách hiệu quả. Những định hướng và lời khuyên của thầy đã giúp em xác định rõ mục tiêu, rèn luyện kỹ năng trình bày, viết báo cáo và quản lý thời gian - những kỹ năng quan trọng cho hành trình học thuật và nghề nghiệp sau này.

Em cũng xin gửi lời cảm ơn chân thành đến quý thầy cô trong Khoa Điện tử - Viễn thông, Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh, những người đã không ngừng nỗ lực giảng dạy, truyền đạt kiến thức và truyền cảm hứng học tập cho sinh viên. Sự tận tâm của quý thầy cô chính là nền tảng vững chắc giúp em hoàn thành khóa luận này.

Bên cạnh đó, em xin cảm ơn các anh chị và bạn bè trong phòng thí nghiệm DESLab đã luôn hỗ trợ, chia sẻ kiến thức, kinh nghiệm và động viên em trong quá trình học tập và nghiên cứu. Những trải nghiệm thực tế và sự gắn bó tại phòng thí nghiệm là hành trang quý báu cho con đường phát triển chuyên môn của em trong tương lai.

Cuối cùng, em xin gửi lời cảm ơn sâu sắc nhất đến gia đình, bạn bè và những người thân yêu - những người luôn ở bên cạnh, ủng hộ và tiếp thêm động lực cho em vượt qua mọi khó khăn. Chính sự yêu thương và niềm tin của họ là nguồn sức mạnh tinh thần to lớn, giúp em vững bước và có được kết quả ngày hôm nay.

Thành phố Hồ Chí Minh, ngày 15 tháng 5 năm 2025

Sinh viên thực hiện

Trịnh Huy Hoàng

LỜI CAM ĐOAN

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

Sinh viên thực hiện

Trịnh Huy Hoàng

TÓM TẮT

Sự bùng nổ của dữ liệu lớn và trí tuệ nhân tạo đang tạo ra những tác động mạnh mẽ lên thị trường phần cứng, đồng thời thúc đẩy nhu cầu về các thiết bị internet vạn vật có khả năng tiết kiệm năng lượng và đạt hiệu năng cao nhằm thực hiện các thuật toán phức tạp. Song song đó, sự phát triển của các thuật toán mã hóa nhẹ cũng được đẩy mạnh do tầm quan trọng ngày càng gia tăng của vấn đề bảo mật thông tin. Khóa luận này đề xuất một kiến trúc hệ thống trên chip (SoC) đa lõi tiên tiến nhằm đáp ứng những yêu cầu trên. Hệ thống bao gồm bốn lõi xử lý đồng nhất RISC-V 64-bit, các lõi gia tốc mật mã như KLEIN, ChaCha và SHA3-512, được kết nối với nhau thông qua kiến trúc mạng trên chip (NoC). Toàn bộ quá trình thiết kế và tổng hợp được thực hiện bằng các công cụ mã nguồn mở, mang lại tính linh hoạt cao và khả năng tùy chỉnh dễ dàng. Kết quả thực nghiệm trên bo mạch FPGA VC707 cho thấy kiến trúc được đề xuất đã vượt trội so với các thiết kế truyền thống sử dụng đơn lõi và phần mềm, đặc biệt trong các tác vụ tính toán phức tạp và xử lý mật mã.

Từ khóa — RISC-V, SoC, đa lõi, mật mã hóa nhẹ, NoC.

ABSTRACT

The rise of big data and Artificial Intelligence is storming the hardware market, boosting demand for Internet of Things devices for energy saving and greater performance to execute complex algorithms. Simultaneously, the development of lightweight encryption algorithms was encouraged by the growing importance of information security. This thesis proposed an advanced multi-core system-on-chip (SoC) architecture to meet the above requirements. The system included four-core 64-bit RISC-V homogeneous, cipher accelerator cores such as KLEIN, ChaCha, and SHA3-512, with Network-on-Chip (NoC) interconnect. The whole design and synthesis process was implemented using open-source tools, which provide high flexibility and customizability. Experimental results on the VC707 FPGA board show that my architecture outperformed traditional single-core and software designs, especially in complex computational tasks and cryptographic processing.

Keywords — RISC-V, SoC, multi-core, lightweight cryptography, NoC.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	v
DANH SÁCH CHỮ VIẾT TẮT	ix
DANH SÁCH CÁC HÌNH	xi
DANH SÁCH CÁC BẢNG	xiii
1 GIỚI THIỆU	1
1.1 Bối cảnh và động lực thực hiện	1
1.2 Mục tiêu nghiên cứu	1
1.3 Phạm vi và đối tượng nghiên cứu	3
1.4 Các công cụ và board mạch phát triển	3
1.4.1 Chisel	3
1.4.2 Chipyard	5
1.4.3 Vivado	6
1.4.4 Board FPGA VC707	7
1.5 Cấu trúc của khóa luận	8
2 TỔNG QUAN NGHIÊN CỨU	9
2.1 Các nghiên cứu liên quan	9
2.1.1 Nghiên cứu về RISC-V ISA	9
2.1.2 Nghiên cứu về framework SoC	9
2.1.3 Nghiên cứu về hệ thống SoC đa lõi	10
2.1.4 Nghiên cứu về Network-on-Chip	11
2.1.5 Nghiên cứu về tích hợp lõi mật mã hóa phần cứng	11
2.2 Tổng kết	12
3 CƠ SỞ LÝ THUYẾT	13
3.1 Quy trình thiết kế số trên FPGA với Chisel	13
3.2 Tổng quan hệ thống	15
3.3 Lõi vi xử lý Rocket	17
3.4 IP ngoại vi	18

3.4.1	DDR Controller	18
3.4.2	Uncore	18
3.4.3	Giao tiếp I/O	21
3.5	Lỗi mật mã hóa nhẹ	22
3.5.1	Mã hóa khối KLEIN	22
3.5.2	Mã hóa dòng ChaCha	23
3.5.3	Hàm băm SHA3-512	24
3.6	Liên kết mạng trên chip	25
3.6.1	Kiến trúc mạng	25
3.6.2	Thuật toán định tuyến	27
3.6.3	Cơ chế điều khiển luồng	28
3.7	Thuật toán nhân ma trận	31
3.8	Sơ đồ phân bổ bộ nhớ	33
4	TRIỂN KHAI HỆ THỐNG TRÊN FPGA	35
4.1	Thiết kế và triển khai SoC trên FPGA bằng ChipYard	35
4.2	Tích hợp các lỗi gia tốc mật mã hóa nhẹ	36
4.3	Triển khai Network-on-Chip bằng Constellation	37
4.4	Phát triển phần mềm	38
4.4.1	Chương trình bootloader	38
4.4.2	Chương trình kiểm tra hệ thống SoC	40
4.4.3	Chương trình kiểm tra các lỗi mật mã hóa	41
4.4.4	Chương trình thuật toán nhân ma trận	43
5	KẾT QUẢ THỬ NGHIỆM	45
5.1	Kết quả về chức năng	45
5.2	Kết quả về hiệu suất	47
5.3	Kết quả về tài nguyên và công suất	49
5.4	Tổng hợp kết quả	51
6	KẾT LUẬN	53
6.1	Kết quả đạt được	53
6.2	Hạn chế và định hướng phát triển	54
	DANH MỤC CÔNG TRÌNH CỦA TÁC GIẢ	55
	TÀI LIỆU THAM KHẢO	57

DANH SÁCH CHỮ VIẾT TẮT

AI Artificial Intelligence.

ASIC Application-Specific Integrated Circuit.

CPU Central Processing Unit.

CSR Control and Status Register.

DSL Domain-Specific Language.

DSP Digital Signal Processing.

FF Flip-Flop.

FPGA Field Programmable Gate Array.

HDL Hardware Description Language.

HPC High Performance Computing.

I/O Input/Output.

IoT Internet of Things.

IP Intellectual Property.

ISA Instruction Set Architecture.

LUT Look-Up Table.

MMIO Memory-Mapped I/O.

NoC Network-on-Chip.

RTL Register-Transfer Level.

SoC System-on-Chip.

DANH SÁCH CÁC HÌNH

1.1	Logo của Chisel	4
1.2	Logo của trình biên dịch sbt	4
1.3	Logo của framework Chipyard [1]	5
1.4	Board VC707	7
3.1	Sơ đồ thiết kế FPGA dùng Chisel	13
3.2	Kiến trúc hệ thống đề xuất	16
3.3	Kiến trúc đường ống của lõi Rocket	17
3.4	Phần chuẩn (các bit từ 15 đến 0) của thanh ghi <code>mip</code>	20
3.5	Phần chuẩn (các bit từ 15 đến 0) của thanh ghi <code>mie</code>	20
3.6	Phần chuẩn (các bit từ 31 đến 0) của thanh ghi <code>mstatus</code>	20
3.7	Kiến trúc hoạt động của <i>PLIC</i> trong hệ thống đa lõi	21
3.8	Thuật toán <i>KeySchedule</i> với độ dài khóa 64-bit	23
3.9	Trạng thái khởi tạo của bộ mã dòng <i>ChaCha</i>	24
3.10	Cấu trúc sponge của thuật toán <i>SHA3-512</i>	25
3.11	Topology trong hệ thống NoC đề xuất	26
3.12	Ví dụ về định tuyến tối thiểu và không tối thiểu	27
3.13	Cấu trúc của messages, packets, flits và phits	28
3.14	Sơ đồ tổ chức của cơ chế điều khiển luồng dựa trên tín dụng	29
3.15	Minh họa quá trình truyền dữ liệu giữa bộ phát và bộ thu	30
3.16	Sơ đồ phân bổ bộ nhớ trong hệ thống đa lõi RISC-V	34
4.1	Cấu trúc tổ chức của một dự án Chipyard	35
4.2	Kiến trúc bộ gia tốc mật mã với giao diện TileLink	36
4.3	Kiến trúc của router	37
4.4	Quy trình khởi động firmware của hệ thống	39
4.5	Lưu đồ chương trình kiểm tra hệ thống SoC	40
4.6	Lưu đồ chương trình nhân ma trận	44
5.1	Kết quả chương trình kiểm tra hệ thống đơn giản	45
5.2	Kết quả thực thi mã hóa khối KLEIN-64	46
5.3	Kết quả thực thi mã hóa dòng ChaCha	46
5.4	Kết quả thực thi hàm băm SHA3-512	47
5.5	So sánh triển khai phần cứng và phần mềm của các lõi gia tốc	47

5.6	Mức sử dụng tài nguyên của hệ thống liên kết	50
5.7	Tổng quan công suất tiêu thụ của hệ thống	50

DANH SÁCH CÁC BẢNG

3.1	Sơ đồ các thanh ghi MSIP của <i>CLINT</i>	19
4.1	Địa chỉ truy cập tới các lỗi mật mã hóa nhẹ	41
4.2	Ảnh xạ thanh ghi cho các tín hiệu của bộ gia tốc mật mã	42
5.1	Kết quả so sánh phép nhân ma trận	48
5.2	Mức sử dụng tài nguyên của hệ thống	49
5.3	Thống kê về công suất theo từng thành phần	51
5.4	Tổng hợp kết quả hệ thống SoC đa lõi	52

CHƯƠNG 1: GIỚI THIỆU

1.1 Bối cảnh và động lực thực hiện

Trong kỷ nguyên của internet vạn vật (IoT), trí tuệ nhân tạo tại biên (Edge AI) và các hệ thống máy tính hiệu năng cao (HPC), nhu cầu về một nền tảng tính toán có thể đáp ứng đồng thời các yêu cầu khắt khe về hiệu năng, khả năng mở rộng và bảo mật đang ngày càng lớn. Điển hình trong các ứng dụng hiện đại như thiết bị y tế thông minh, xe tự hành, hay hệ thống điều khiển công nghiệp thời gian thực đều phải xử lý khối lượng dữ liệu lớn với yêu cầu tính toán song song và truyền tải dữ liệu nhanh chóng, trong khi vẫn đảm bảo an toàn trước các mối đe dọa tấn công bảo mật ngày càng tinh vi và khó đoán.

Trong bối cảnh đó, kiến trúc System-on-Chip (SoC) nổi lên như một giải pháp chủ đạo, nhờ khả năng tích hợp các chức năng tính toán, ngoại vi và bảo mật trong cùng một vi mạch duy nhất. Đặc biệt, với sự phát triển của kiến trúc tập lệnh mã nguồn mở RISC-V, việc triển khai các hệ thống SoC với nhiều lõi xử lý song song (multi-core) trở nên linh hoạt và tùy biến hơn. Đồng thời các thành phần trong hệ thống có thể được kết nối hiệu quả thông qua Network-on-Chip (NoC) nhằm giảm độ trễ và tắt nghẽn băng thông khi truyền dữ liệu nội bộ giữa các lõi xử lý hoặc giữa các lõi và bộ nhớ chung.

Tuy nhiên, một vấn đề nổi bật hiện nay là phần lớn các thiết kế SoC vẫn thiếu sự kết hợp giữa hiệu năng và bảo mật. Trong nhiều công trình, việc tích hợp một CPU với một hoặc hai lõi mã hóa không đủ để đáp ứng các ứng dụng yêu cầu bảo mật cao hoặc môi trường xử lý song song phức tạp. Đồng thời, nếu chỉ dựa vào triển khai bảo mật bằng phần mềm, hệ thống sẽ tiêu tốn nhiều tài nguyên tính toán, làm giảm hiệu năng và tăng mức tiêu thụ năng lượng.

Từ đó, có thể thấy rằng việc thiết kế một hệ thống SoC đa lõi RISC-V với mạng NoC hiệu quả, đồng thời tích hợp thêm các lõi bảo mật chuyên dụng là một hướng nghiên cứu cần thiết và tiềm năng để xây dựng nền tảng cho các hệ thống tính toán hiện đại trong tương lai.

1.2 Mục tiêu nghiên cứu

Mục tiêu của đề tài là thiết kế và phát triển một hệ thống SoC sử dụng kiến trúc đa lõi của RISC-V, kết hợp với liên kết NoC và tích hợp thêm các lõi gia tốc mật mã hóa nhẹ. Hệ thống đạt được sẽ đáp ứng các yêu cầu hiệu năng cao cho các ứng dụng

tính toán song song đồng thời đảm bảo bảo mật thông tin hiệu quả. Cuối cùng, hệ thống sẽ được triển khai trên nền tảng FPGA với mục tiêu kiểm tra và đánh giá chức năng, hiệu suất, tài nguyên và công suất hệ thống.

Cụ thể, các mục tiêu chi tiết của đề tài bao gồm:

1. Thiết kế và tích hợp hệ thống SoC:

- Xây dựng hệ thống SoC đa lõi đồng nhất trên kiến trúc RISC-V, trong đó tất cả các lõi xử lý đều sử dụng cùng một bộ các tập lệnh thường dùng trong RISC-V ISA.
- Tích hợp đầy đủ các thành phần ngoại vi cần thiết như bộ nhớ cache, bộ điều khiển DDR (DDR controller) và các ngoại vi hỗ trợ khác để đảm bảo hệ thống hoạt động hiệu quả khi triển khai trên FPGA.
- Thiết kế và triển khai hệ thống bus phân tầng kết hợp với liên kết giao tiếp NoC để kết nối các lõi RISC-V và các thành phần khác trong hệ thống.

2. Tích hợp các lõi mật mã hóa nhẹ:

- Tích hợp đa dạng các lõi mật mã hóa nhẹ như mã hóa khối KLEIN, mã hóa dòng ChaCha và hàm băm SHA3-512.
- Các lõi này sẽ được kiểm tra nhằm đảm bảo hoạt động đúng chức năng và có thể được điều khiển thông qua các lõi CPU của hệ thống.

3. Phát triển firmware:

- Xây dựng chương trình kiểm tra khả năng hoạt động của hệ thống và các lõi CPU.
- Xây dựng chương trình kiểm tra chức năng và so sánh tốc độ hoạt động của các lõi gia tốc mật mã hóa với phiên bản triển khai phần mềm.
- Xây dựng chương trình thuật toán nhân ma trận dựa trên khả năng xử lý song song của các lõi CPU.

4. Đánh giá hiệu năng hệ thống:

- Đánh giá tài nguyên và công suất: Đo lường và đánh giá mức độ sử dụng tài nguyên logic và công suất tiêu thụ trên FPGA khi triển khai hệ thống. Các yếu tố đánh giá bao gồm sử dụng IP bộ nhớ ngoài và sử dụng các thành phần phần cứng (lõi RISC-V, hệ thống bus, lõi mật mã hóa).

- Đánh giá hiệu năng: Kiểm tra tốc độ xử lý của các lõi mã hóa trong việc thực hiện các thuật toán bảo mật, đồng thời so sánh hiệu năng của hệ thống đa lõi so với hệ thống đơn lõi trong việc thực hiện thuật toán nhân ma trận.

1.3 Phạm vi và đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài tập trung vào phát triển kiến trúc SoC, vi xử lý RISC-V, liên kết giao tiếp NoC và các lõi gia tốc mật mã hóa nhẹ.

Phạm vi nghiên cứu của đề tài này được xác định như sau:

- Đề tài sử dụng Chisel để thiết kế và mô tả các thành phần trong hệ thống SoC.
- Hệ thống sẽ được mô phỏng và kiểm thử trên nền tảng FPGA.
- Kiểm tra chức năng và đánh giá hiệu năng các thành phần sẽ được thực hiện thông qua các firmware tự thiết kế.
- Đề tài cũng bao gồm việc phát triển một bootloader để khởi động hệ thống, nạp phần mềm từ SD Card vào bộ nhớ và đảm bảo hệ thống hoạt động đúng khi bật nguồn.
- Phạm vi nghiên cứu chỉ tập trung vào kiến trúc hệ thống, bao gồm các thành phần vi xử lý, NoC và bảo mật phần cứng. Các khía cạnh thiết kế khác như layout vật lý của các thành phần phần cứng sẽ không được thực hiện.
- Các lõi mật mã hóa nhẹ và các ngoại vi khác (bộ nhớ, bộ điều khiển) sẽ được tích hợp ở mức ứng dụng. Việc nghiên cứu phương pháp thiết kế lõi mật mã hóa và tối ưu các thuật toán phần cứng không nằm trong phạm vi đề tài.
- Các thành phần như hệ điều hành, driver ngoại vi và các phần mềm hệ thống phức tạp khác cũng không thuộc phạm vi nghiên cứu của đề tài.

1.4 Các công cụ và board mạch phát triển

1.4.1 Chisel

Thay vì sử dụng ngôn ngữ Verilog HDL truyền thống để thiết kế phần cứng, nghiên cứu này sử dụng Chisel - một ngôn ngữ sinh phần cứng hiện đại, được phát triển bởi đại học California, Berkeley (UC Berkeley). Chisel được triển khai dưới

dạng một Domain-Specific Language (DSL) của ngôn ngữ Scala, cho phép mô tả các thiết kế phần cứng ở mức độ trừu tượng và module hóa cao, phù hợp với các hệ thống lớn như vi xử lý và các lõi xử lý tín hiệu số.



Hình 1.1: Logo của Chisel

Khác với các HDL truyền thống, Chisel cho phép sử dụng các kỹ thuật lập trình hướng đối tượng và lập trình hàm - nhờ kế thừa toàn bộ sức mạnh của Scala. Điều này mang lại khả năng tạo ra các bộ sinh mã (code generators) có tính tham số hóa cao, giúp việc thiết kế phần cứng trở nên linh hoạt và dễ bảo trì hơn, đồng thời cũng mang lại tính tái sử dụng cao. Ngoài ra, Chisel còn giúp rút ngắn chu kỳ phát triển nhờ khả năng mô tả phần cứng ở mức cao và sinh mã Verilog tuân theo chuẩn công nghiệp.

Trong hệ sinh thái của Chisel, công cụ sbt (simple build tool) đóng vai trò quan trọng trong việc quản lý dự án, biên dịch mã nguồn và hỗ trợ quy trình phát triển phần cứng. sbt được xây dựng dành riêng cho Scala, hỗ trợ xử lý song song, kiểm tra lỗi tại thời điểm biên dịch và tích hợp dễ dàng các công cụ mô phỏng, kiểm thử cũng như sinh mã Verilog từ thiết kế Chisel. Với hơn 100 plugin mã nguồn mở và khả năng mở rộng cao, sbt giúp đơn giản hóa quy trình phát triển, hỗ trợ các phương pháp hiện đại như tích hợp liên tục (continuous integration) và kiểm thử tự động.



Hình 1.2: Logo của trình biên dịch sbt

Hiện nay, Chisel đang được cộng đồng mã nguồn mở phát triển mạnh mẽ và liên tục cập nhật. Một trong những minh chứng tiêu biểu cho khả năng ứng dụng thực tiễn của Chisel là dự án Rocket Chip - một bộ sinh kiến trúc SoC có thể mở rộng và tùy chỉnh, được thiết kế dựa trên tập lệnh RISC-V. Rocket Chip cho thấy tiềm năng lớn của Chisel trong việc phát triển các hệ thống vi xử lý hiện đại, linh hoạt và có khả năng tái sử dụng cao.

Nhằm mở rộng và phát triển hơn nữa năng lực của Rocket Chip, dự án ChipYard được tạo ra và là một framework mã nguồn mở dành cho phát triển SoC được cộng đồng đón nhận và phát triển tích cực nhất.

1.4.2 Chipyard

Chipyard là một framework mã nguồn mở cũng được phát triển tại UC Berkeley, với mục tiêu tích hợp toàn diện các công cụ thiết kế, mô phỏng và triển khai hệ thống SoC. Đây là kết quả của quá trình phát triển và tích hợp giữa Chisel, Rocket Chip, và Rocket Core - lõi xử lý trung tâm của Rocket Chip.



Hình 1.3: Logo của framework Chipyard [1]

Chipyard tận dụng sức mạnh của Chisel, bộ biến đổi trung gian FIRRTL, các bộ sinh mã như Rocket Chip Generator, cùng nhiều công cụ hỗ trợ khác để tạo ra một môi trường phát triển SoC RISC-V linh hoạt, dễ mở rộng. Nền tảng này cung cấp một thư viện phong phú gồm các lõi xử lý như Rocket Core, BOOM, Ariane, các bộ tăng tốc phần cứng như Hwacha, Gemmini, SHA3, cùng với hệ thống bộ nhớ, ngoại vi và các công cụ cần thiết để xây dựng một hệ thống SoC đầy đủ tính năng.

Chipyard hỗ trợ nhiều quy trình phát triển khác nhau, bao gồm:

- Giả lập phần cứng tốc độ cao bằng FPGA thông qua FireSim.
- Mô phỏng RTL phần mềm.
- Dùng FPGA kiểm thử phần cứng (FPGA Prototyping).
- Triển khai ASIC tự động với Hammer.

Thông qua sự tích hợp chặt chẽ giữa các công cụ này, Chipyard mang đến một quy trình phát triển đồng bộ, tuần hoàn và hiệu quả - từ mô tả thiết kế ở mức độ trừu tượng cao cho đến triển khai vật lý ASIC. Khai thác tính tích hợp và đồng bộ cao của Chipyard nhằm phát triển không chỉ các kiến trúc CPU đơn lõi và đa lõi, mà còn mở rộng sang các hệ thống tích hợp toàn diện, bao gồm bộ nhớ, bộ tăng tốc phần cứng và ngoại vi.

Việc sử dụng Chipyard trong nghiên cứu giúp tăng tính linh hoạt, khả năng tùy biến và hiệu quả trong quá trình thiết kế - từ các kiến trúc đơn giản đến phức tạp. Đồng thời, nền tảng này cũng hỗ trợ đánh giá hiệu năng, tối ưu hóa mức tiêu thụ năng lượng và định hướng phát triển các ứng dụng phần cứng tùy chỉnh trong tương lai.

1.4.3 Vivado

Trong lĩnh vực FPGA, hai hãng nổi bật nhất hiện nay là Xilinx (thuộc AMD) và Altera (thuộc Intel). Cả hai đều cung cấp các dòng FPGA hiệu suất cao, phù hợp với nhiều ứng dụng. Tuy nhiên, Xilinx nổi bật với các dòng sản phẩm hướng đến các ứng dụng phức tạp như AI, big data và truyền thông, cùng với bộ công cụ thiết kế Vivado Design Suite mạnh mẽ, tích hợp tốt và hỗ trợ tối ưu hóa tự động. Trong khi đó, Altera thường tập trung vào các ứng dụng điều khiển công nghiệp và xử lý tín hiệu số với công cụ Quartus Prime.

Vivado đã ra mắt từ năm 2012 và được thiết kế tối ưu cho các dòng FPGA hiện đại như Artix-7, Kintex-7, Virtex-7 và đặc biệt là dòng Versal. Vivado không chỉ hỗ trợ thiết kế RTL, mà còn tích hợp các công cụ cao cấp như High-Level Synthesis (HLS), Logic Analyzer và Static Timing Analysis (STA), hỗ trợ cả thiết kế phần cứng lẫn kiểm tra chức năng.

Trong đó, phiên bản Vivado 2021.2 ML Edition là phiên bản cập nhật tiên tiến với các tính năng nổi bật sau:

- Tối ưu hóa bằng Machine Learning, giúp tăng hiệu suất tổng hợp và định tuyến.
- Hỗ trợ Dynamic Function eXchange (DFX), cho phép thay đổi cục bộ thiết kế mà không dừng toàn bộ hệ thống.
- HLS cải tiến, hỗ trợ chuyển đổi mã C/C++ sang phần cứng nhanh chóng.
- Phân tích logic và thời gian nâng cao, thích hợp với các thiết kế hiệu năng cao.
- Tối ưu cho dòng FPGA Versal, tích hợp khả năng xử lý AI và truyền thông băng thông cao.

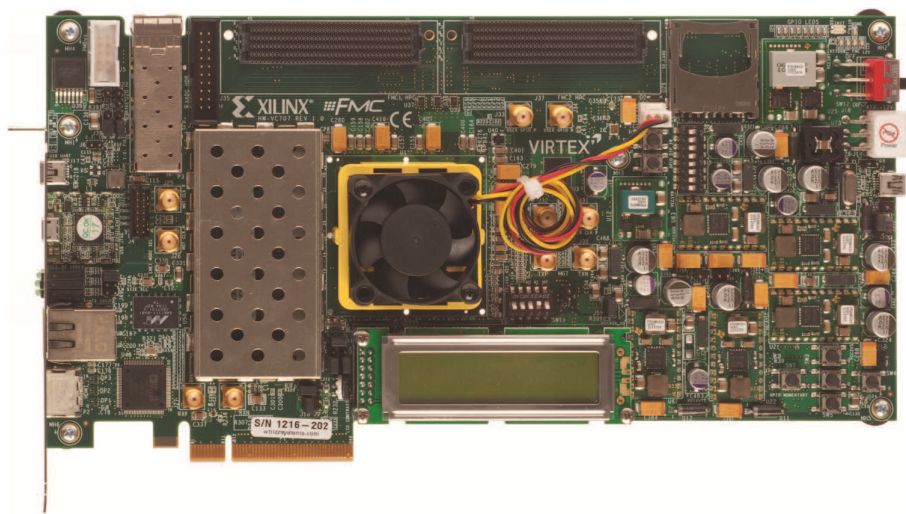
Vivado 2021.2 ML còn cung cấp một môi trường phát triển tích hợp đầy đủ, từ viết mã, tổng hợp, mô phỏng, kiểm thử đến triển khai. Công cụ Simulator, IBERT và DRC giúp đảm bảo thiết kế đạt độ chính xác và ổn định cần thiết trước khi triển khai.

Trong khuôn khổ của khóa luận, board FPGA VC707 sẽ được sử dụng để triển khai thiết kế hệ thống bảo mật SoC nhờ vào khả năng hỗ trợ tốt cho Vivado, nhiều tài nguyên phần cứng và phù hợp với cơ sở vật chất hiện tại của phòng thí nghiệm.

1.4.4 Board FPGA VC707

Board VC707 sử dụng FPGA dòng Virtex-7 với hiệu suất cao và tài nguyên phong phú, đáp ứng tốt cho các ứng dụng cần sức mạnh xử lý lớn và khả năng kết nối tốc độ cao. Khi phát triển các thiết kế ứng dụng có quy mô lớn, phức tạp, yêu cầu thiết kế đa lõi hoặc xử lý nhiều tác vụ đồng thời, VC707 là một lựa chọn phù hợp. Các thông số kỹ thuật cơ bản của VC707 bao gồm:

- FPGA: Virtex-7 XC7VX485T với 485,760 logic cells.
- DSP slices: 2,800 DSP slices, hỗ trợ các ứng dụng xử lý tín hiệu số chuyên sâu.
- Bộ nhớ: 37,080 Kbits RAM tích hợp, cung cấp dung lượng bộ nhớ lớn cho xử lý dữ liệu.
- Giao diện tốc độ cao: Hỗ trợ đến 16 giao diện GTP Transceiver cho truyền dữ liệu tốc độ cao.
- PCIe Gen2: Tích hợp giao diện PCI Express Gen2 x8, cho phép truyền dữ liệu với băng thông lớn.
- DDR3 SDRAM: Trang bị 1GB DDR3 SDRAM, phù hợp cho các ứng dụng AI, học máy và xử lý dữ liệu lớn.
- Ethernet: Hỗ trợ truyền dữ liệu mạng với băng thông 10/100/1000 Mbps.



Hình 1.4: Board VC707

1.5 Cấu trúc của khóa luận

Khóa luận này được bố cục thành sáu chương, trình bày tuần tự từ bối cảnh nghiên cứu, tổng quan lý thuyết, thiết kế, triển khai thực nghiệm đến phân tích, đánh giá kết quả và kết luận. Cụ thể:

- **Chương 1 – Giới thiệu:** Trình bày bối cảnh nghiên cứu, động lực thực hiện đề tài, mục tiêu, phạm vi, các công cụ nghiên cứu được sử dụng và tóm lược cấu trúc toàn khóa luận.
- **Chương 2 – Tổng quan nghiên cứu:** Khảo sát và phân tích các công trình nghiên cứu liên quan đến kiến trúc RISC-V, các framework thiết kế SoC, kiến trúc đa lõi, NoC và các phương pháp tích hợp lõi mã hóa nhẹ, từ đó chỉ ra khoảng trống nghiên cứu mà khóa luận tập trung giải quyết.
- **Chương 3 – Cơ sở lý thuyết:** Trình bày nền tảng lý thuyết phục vụ cho quá trình nghiên cứu, bao gồm quy trình thiết kế số trên FPGA sử dụng Chisel, kiến trúc SoC đa lõi, chi tiết lõi Rocket, các IP ngoại vi, ba thuật toán mật mã hóa nhẹ (KLEIN, ChaCha, SHA3-512), kiến thức về thiết kế NoC vùng với thuật toán nhân ma trận.
- **Chương 4 – Triển khai hệ thống trên FPGA:** Mô tả chi tiết quá trình cấu hình hệ thống SoC thông qua Chipyard, tích hợp các lõi gia tốc mật mã, xây dựng mạng NoC, lập trình firmware boot và phát triển các phần mềm thử nghiệm cũng như phép toán nhân ma trận.
- **Chương 5 – Kết quả thực nghiệm:** Trình bày kết quả kiểm tra chức năng và hiệu suất của hệ thống SoC, phân tích so sánh giữa phần cứng và phần mềm, thống kê mức sử dụng tài nguyên phần cứng, năng lượng, đồng thời tổng hợp kết quả và thông số chính của hệ thống.
- **Chương 6 – Kết luận:** Tóm tắt những đóng góp chính của khóa luận, đánh giá những hạn chế còn tồn tại và đề xuất hướng nghiên cứu mở rộng trong tương lai như tăng số lượng lõi xử lý, hỗ trợ cơ chế coherence, tối ưu hóa năng lượng và khả năng hiện thực hóa trên ASIC.

Bên cạnh đó, khóa luận còn đính kèm **Danh mục công trình khoa học của tác giả** và phần **Tài liệu tham khảo** ở cuối nhằm cung cấp thông tin chi tiết cho việc tra cứu.

CHƯƠNG 2: TỔNG QUAN NGHIÊN CỨU

2.1 Các nghiên cứu liên quan

Phần này trình bày tổng quan về các công trình nghiên cứu có liên quan đến đề tài, bao gồm các chủ đề như: kiến trúc tập lệnh RISC-V, framework hỗ trợ thiết kế SoC, hệ thống SoC đa lõi, mạng trên chip và các lõi gia tốc mật mã hóa nhẹ. Các hướng nghiên cứu này cung cấp nền tảng quan trọng để phát triển một hệ thống SoC hiệu năng cao và bảo mật như mục tiêu đề tài đã đặt ra.

2.1.1 Nghiên cứu về RISC-V ISA

Đầu tiên là RISC-V, một kiến trúc tập lệnh mã nguồn mở đã không còn xa lạ gì với cộng đồng nghiên cứu và công nghiệp. Theo báo cáo của Semico Research [26], RISC-V được dự báo sẽ trở thành một thành phần quan trọng trong các thiết kế CPU trong tương lai. Cụ thể, tổng số vi xử lý RISC-V được tiêu thụ toàn cầu dự kiến sẽ đạt 62,4 tỷ đơn vị vào năm nay, với các sản phẩm chính tập trung vào SoC đơn giản, SoC đa lõi và các thiết kế dựa trên FPGA. Ngoài các tập lệnh cơ bản sẵn có, RISC-V còn cho phép mở rộng tập lệnh để đáp ứng các yêu cầu chuyên biệt của từng ứng dụng.

Trong bài khảo sát của Enfang Cui và cộng sự [4], tác giả đã phân tích chi tiết các tập lệnh mở rộng (extension) của RISC-V trong nhiều bối cảnh ứng dụng như IoT [17], AI [7], GPU [31] và HPC [2]. Bài báo cũng chỉ ra nhu cầu phát triển thêm các extension phù hợp với từng lĩnh vực, ví dụ như vi xử lý nhúng cần được bổ sung các tập lệnh hỗ trợ tối ưu năng lượng và xử lý dữ liệu thời gian thực.

2.1.2 Nghiên cứu về framework SoC

Song song với sự phát triển mạnh mẽ của kiến trúc CPU RISC-V, nhiều framework hỗ trợ tích hợp và phát triển SoC đã được giới thiệu nhằm đơn giản hóa quá trình thiết kế và hiện thực hóa hệ thống. Trong số đó, nổi bật nhất là Chipyard - một framework mã nguồn mở được sử dụng trong đề tài này - và PULP (Parallel Ultra-Low Power Platform) - một nền tảng phổ biến tại Châu Âu. PULP [25] là một nền tảng mã nguồn mở có khả năng mở rộng cao, hỗ trợ nhiều loại thiết kế từ vi xử lý đơn lõi tiết kiệm năng lượng như PULPino [10], đến các hệ thống đa lõi cho thiết bị IoT như MrWolf [22] và thậm chí là nền tảng many-core cho các ứng dụng tính toán hiệu suất cao. Với khả năng hỗ trợ đa dạng từ kiến trúc phần cứng đến phần mềm,

PULP là một framework có tiềm năng lớn trong việc phát triển các SoC đa lõi hiệu năng cao nhưng tiêu thụ năng lượng thấp - hướng đi phù hợp với nhiều ứng dụng nhúng hiện đại.

Mặt khác, Chipyard [1] là một nền tảng tích hợp toàn diện được xây dựng dựa trên các thành phần lõi như Rocket Chip Generator, hỗ trợ mô phỏng, tổng hợp phần cứng và tích hợp phần mềm thông qua chuỗi công cụ hiện đại. Do đã được giới thiệu trước đó nên phần này sẽ không nhắc lại nữa.

2.1.3 Nghiên cứu về hệ thống SoC đa lõi

Bằng cách sử dụng những framework này, kiến trúc SoC đa lõi được nghiên cứu mạnh mẽ để trở thành nền tảng cốt lõi cho các ứng dụng từ nhúng đến trí tuệ nhân tạo và điện toán biên (edge computing). Các công trình nghiên cứu gần đây đã tập trung vào hai hướng chính: kiến trúc SoC đa lõi đồng nhất (homogeneous) và không đồng nhất (heterogeneous), sử dụng lõi RISC-V làm thành phần xử lý trung tâm.

Trong [33], Yuan và các cộng sự đề xuất một hệ thống SoC đa lõi đồng nhất sử dụng các bộ xử lý dấu chấm động (Floating-Point Processors) với kiến trúc NoC dạng lưới 2D. Kết quả thực nghiệm trên FPGA cho thấy hệ thống có khả năng tăng tốc tuyến tính khi mở rộng số lượng lõi lên 8, đặc biệt hiệu quả trong bài toán nhân ma trận với ma trận kích thước lớn (512×512), chứng minh năng lực xử lý song song mạnh mẽ của hệ thống.

Bên cạnh đó, Kieu-Do-Nguyen và các cộng sự [21] giới thiệu phương pháp thiết kế đồng thời phần cứng/phần mềm (hardware/software co-design) cho hệ thống đa lõi RISC-V nhằm tối ưu hóa xử lý song song và quản lý tài nguyên. Hệ thống được triển khai trên nền tảng FPGA với 4 lõi Rocket RV64 đồng nhất, kết hợp với một bộ quản lý luồng (thread manager) và bộ nhớ cache lai (hybrid L1 cache). Nhờ kiến trúc phân tầng bus và phương pháp quản lý luồng hiệu quả, hệ thống đạt được hiệu năng cao hơn 400% so với kiến trúc đơn lõi trong các bài toán như nhân ma trận, tích chập 2D và CRC-32. Đặc biệt, phần mở rộng phần cứng chỉ chiếm chưa đến 1% tài nguyên FPGA, cho thấy hiệu quả vượt trội với chi phí phần cứng thấp.

Một nghiên cứu khác của Gonzalez và các cộng sự trong [14] trình bày một hệ thống SoC đa lõi không đồng nhất sử dụng lõi RISC-V, triển khai trên công nghệ 22nm FinFET của Intel. Hệ thống bao gồm một lõi BOOM [35] và một lõi Rocket [9], kết nối với các bộ gia tốc phần cứng như Hwacha vector accelerator [23] và Gemmini systolic array DNN accelerator [11]. Thiết kế này cho thấy khả năng đạt hiệu suất đến 106.1 GOPS/W và tốc độ cải thiện lên tới 1297.2 lần so với một lõi Rocket đơn

thuần khi thực hiện bài toán nhân ma trận. Kiến trúc này còn hỗ trợ phân chia miền clock, cache chia sẻ L2 1MiB và giao tiếp ngoại vi đa dạng, thể hiện tính mở rộng và khả năng tùy biến cao của SoC RISC-V khi kết hợp các lõi tính toán và gia tốc đặc thù.

2.1.4 Nghiên cứu về Network-on-Chip

Với sự phát triển của các hệ thống SoC đa lõi ngày càng phức tạp, kết nối giữa các lõi xử lý và các lõi bộ nhớ trở thành nút thắt chính về hiệu năng, độ trễ và tiêu thụ năng lượng. Trong bối cảnh đó, Network-on-Chip (NoC) đã nổi lên như một giải pháp thay thế hiệu quả cho kiến trúc bus truyền thống vốn thiếu khả năng mở rộng.

Trong cuốn sách On-Chip Networks (2017), Natalie Enright Jerger và các đồng tác giả [15] đã trình bày toàn diện các khái niệm cơ bản, kiến trúc phần cứng và chiến lược thiết kế của NoC. Tác giả cũng giải thích rõ ràng các thuật ngữ quan trọng trong lĩnh vực này, đồng thời phân tích chi tiết các kiến trúc NoC từ dạng cổ điển như 2D mesh, torus đến các cấu trúc tiên tiến hơn như fat-tree hay butterfly.

Các yếu tố chính trong thiết kế NoC được đề cập bao gồm: kiến trúc mạng (topology), thuật toán định tuyến (routing algorithm), cơ chế điều khiển luồng (flow control) và kiến trúc router. Ngoài ra, cuốn sách cũng nhấn mạnh vai trò của Quality of Service (QoS) và các cơ chế hỗ trợ tính mở rộng module hóa, giúp hệ thống dễ dàng tích hợp thêm các IP mới mà không làm gián đoạn tổng thể hệ thống.

Bên cạnh phần lý thuyết, tác giả còn khảo sát các hướng nghiên cứu và thiết kế SoC sử dụng NoC đang được áp dụng thực tế, đồng thời đưa ra các định hướng phát triển trong tương lai. Đây cũng chính là nguồn tài liệu nền tảng quan trọng để đề tài này tham khảo và ứng dụng trong việc thiết kế NoC cho hệ thống SoC đa lõi tích hợp bảo mật.

2.1.5 Nghiên cứu về tích hợp lõi mật mã hóa phần cứng

Theo khảo sát tổng hợp của Tao Lu trong [18], các hướng bảo mật cho hệ thống RISC-V bao gồm: (i) mở rộng tập lệnh để hỗ trợ mã hóa (ISA extension); (ii) tích hợp phần cứng chuyên dụng (cryptographic accelerators); và (iii) kiến trúc bảo vệ bộ nhớ và giao tiếp. Trong đó, các lõi mã hóa phần cứng nhẹ (lightweight crypto cores) là lựa chọn được quan tâm nhiều nhất để bảo vệ dữ liệu truyền giữa các thành phần trong SoC mà vẫn giữ được hiệu quả tính toán và khả năng triển khai trên FPGA hoặc ASIC.

Một nghiên cứu tiêu biểu là của Khai-Minh Ma và cộng sự [19], trong đó nhóm tác giả đã xây dựng một hệ thống SoC dựa trên lõi CPU RISC-V 32-bit, tích hợp bốn lõi mã hóa nhẹ: PRINCE, PRESENT-80, ChaCha và SHA3-512, được thiết kế tối ưu hóa độ trễ và diện tích phần cứng. Hệ thống được triển khai trên FPGA với mức sử dụng tài nguyên tương đối thấp, bao gồm 11.830 LUTs và 9.552 FFs. Nghiên cứu này cũng cho thấy rằng việc tích hợp phần cứng mật mã làm giảm gánh nặng xử lý trên CPU và tăng tốc độ mã hóa gấp nhiều lần so với thực thi phần mềm.

Bên cạnh đó, El-Hajj và các cộng sự [8] đã thực hiện một phân tích toàn diện 119 thuật toán mã hóa nhẹ trên các nền tảng phần cứng hạn chế như Arduino và Raspberry Pi. Kết quả cho thấy một số thuật toán như LEA, HIGHT, XTEA và Speck đạt hiệu suất mã hóa cao với chi phí bộ nhớ và năng lượng thấp. Đặc biệt, các ứng viên vòng 2 của cuộc thi NIST LWC như GIFT-COFB, Xoodyak và Schwaemm cho thấy khả năng triển khai hiệu quả về mặt thời gian và năng lượng. Nghiên cứu cũng cung cấp một hệ thống đo lường đầy đủ cho các chỉ số như kích thước mã lệnh, RAM/ROM sử dụng, độ trễ, throughput và năng lượng tiêu thụ, góp phần định hướng lựa chọn thuật toán tối ưu khi triển khai trên phần cứng.

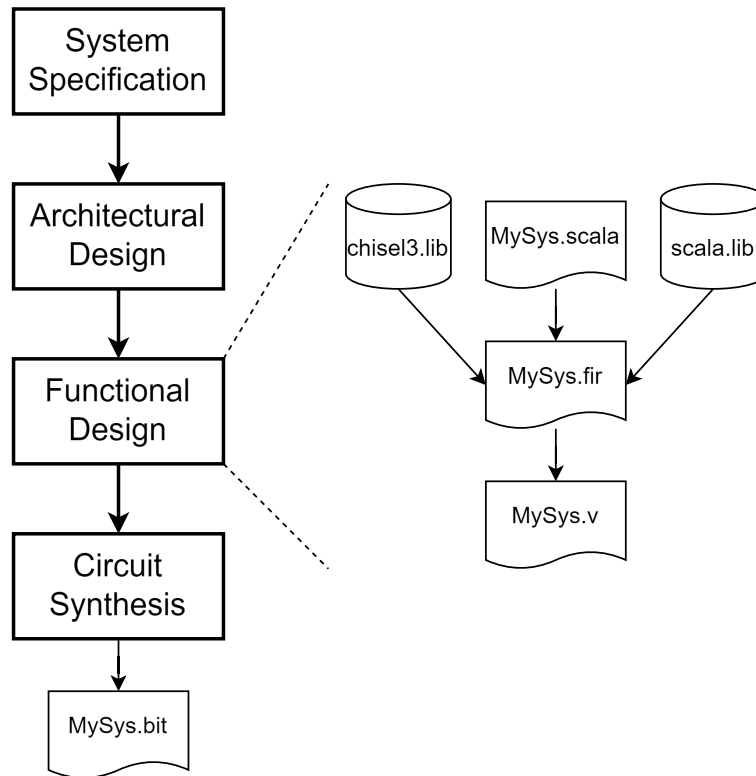
2.2 Tổng kết

Tổng thể, các công trình nghiên cứu trên cho thấy xu hướng thiết kế SoC hiện đại là hướng tới sự kết hợp giữa đa lõi, các lõi gia tốc mật mã hóa riêng biệt và kết nối hiệu quả bằng NoC. Đề tài này kế thừa và tổng hợp các hướng nghiên cứu trên bằng cách triển khai kiến trúc SoC đa lõi RISC-V đồng nhất, liên kết thông qua NoC và đồng thời tích hợp thêm các lõi gia tốc mật mã hóa nhẹ. Hệ thống được triển khai trên FPGA và đánh giá thông qua firmware tự thiết kế.

CHƯƠNG 3: CƠ SỞ LÝ THUYẾT

3.1 Quy trình thiết kế số trên FPGA với Chisel

Khóa luận này được triển khai thông qua hai quy trình chính: thiết kế phần cứng và phát triển phần mềm. Trong đó, thiết kế phần cứng được thực hiện trước, tuân theo quy trình thiết kế mạch số được minh họa trong Hình 3.1 với các bước như sau:



Hình 3.1: Sơ đồ thiết kế FPGA dùng Chisel

Bước 1: Xác định đặc tả kỹ thuật của hệ thống: Ở bước này, bộ khung sơ khai của hệ thống SoC được xây dựng, đồng thời các thông số kỹ thuật cần đạt được cũng được xác định rõ. Cụ thể bao gồm: số lượng lõi CPU, các ngoại vi cần tích hợp, các lõi gia tốc mật mã và kiến trúc kết nối được sử dụng. Ngoài ra, kích thước bộ nhớ đệm cũng được lựa chọn tại bước này.

Bước 2: Thiết kế kiến trúc hệ thống: Nền tảng Chipyard đã cung cấp hầu hết các phương pháp và công cụ cần thiết để xây dựng hệ thống SoC, bao gồm cả hỗ trợ kết nối bus và mạng NoC. Ở bước này, khóa luận lựa chọn phương án tích hợp các lõi mã hóa phần cứng vào hệ thống SoC thông qua cơ chế Memory-Mapped I/O (MMIO) - một phương thức kết nối phổ biến giúp CPU truy cập và điều khiển các IP ngoại vi thông qua không gian địa chỉ bộ nhớ.

Bước 3: Mô tả phần cứng bằng ngôn ngữ Chisel: Khác với quy trình truyền thống sử dụng Verilog, khóa luận sử dụng dụng Chisel - ngôn ngữ sinh phần cứng hướng đối tượng dựa trên Scala, do đó quy trình mô tả phần cứng sẽ được thực hiện như sau:

1. Framework Chipyard được tải về từ GitHub và cài đặt trên hệ điều hành Linux, kèm theo các thư viện và công cụ phụ trợ cần thiết để tạo nên môi trường thiết kế đầy đủ.
2. Tiến hành cấu hình hệ thống SoC bảo mật đa lõi thông qua các tập tin cấu hình của Chisel.
3. Sau khi cấu hình hoàn chỉnh, hệ thống được biên dịch từ Chisel sang Verilog để phục vụ bước tổng hợp sau cùng.

Bước 4: Tổng hợp phần cứng và tạo bitstream: Sau khi thu được các file Verilog, hệ thống được tổng hợp bằng công cụ Xilinx Vivado phiên bản 2024.2, nhằm tạo ra file bitstream (*MySystem.bit*) để có thể nạp xuống FPGA và tiến hành kiểm tra chức năng.

Sau khi hoàn thành quá trình thiết kế và xây dựng phần cứng, bước tiếp theo là phát triển phần mềm điều khiển, bao gồm các firmware để kiểm tra chức năng của các thành phần tích hợp trong hệ thống SoC. Các phần mềm này đảm nhiệm việc:

1. Kiểm tra hoạt động chính xác của các lõi mã hóa phần cứng được tích hợp.
2. Đánh giá khả năng hoạt động độc lập của các lõi CPU RISC-V.
3. Thực thi thuật toán nhân ma trận để kiểm chứng khả năng xử lý song song của hệ thống đa lõi.

Để đơn giản hóa quá trình lập trình và biên dịch, đề tài sử dụng lại hai file linker (*htif.ld*) và startup (*crt0.S*) từ thư viện libgloss [24] do nhóm phát triển ucb-bar cung cấp, đồng thời cấu hình lại các tệp *Makefile* tương ứng cho từng ứng dụng. Quy trình phát triển phần mềm được thực hiện qua các bước sau:

Bước 1: Cài đặt môi trường phát triển và toolchain cho RISC-V: Tiến hành thiết lập toolchain RISC-V, bao gồm trình biên dịch (*64-unknown-elf-gcc*), trình liên kết (*ld*) và các công cụ phụ trợ (*objcopy*, *spike*, *pk*, v.v.), nhằm phục vụ quá trình biên dịch và mô phỏng chương trình trên hệ thống SoC đã xây dựng.

Bước 2: Cấu hình linker, startup và tệp biên dịch tự động: Sử dụng lại file linker script và file startup từ bộ thư viện mã nguồn mở libgloss, sau đó cấu hình *Makefile* cho từng firmware, đảm bảo quá trình biên dịch được tự động hóa và tương thích với địa chỉ bộ nhớ của hệ thống phần cứng.

Bước 3: Lập trình các firmware kiểm thử: Phát triển các chương trình chính tương ứng với các mục tiêu kiểm thử khác nhau:

- Firmware 1: Kiểm tra hoạt động chức năng của các lõi mã hóa phần cứng thông qua các phép thử đầu vào - đầu ra tiêu chuẩn (test vector).
- Firmware 2: Kiểm tra khả năng hoạt động độc lập và thực thi lệnh của từng lõi RISC-V trong hệ thống.
- Firmware 3: Thực thi thuật toán nhân ma trận để đánh giá khả năng hoạt động song song và mức độ tương tác giữa các lõi CPU thông qua kết nối NoC.

Trong số ba bước nêu trên, Bước 1 chỉ cần thực hiện một lần duy nhất, trong khi Bước 2 và Bước 3 được thực hiện riêng biệt cho từng firmware cụ thể.

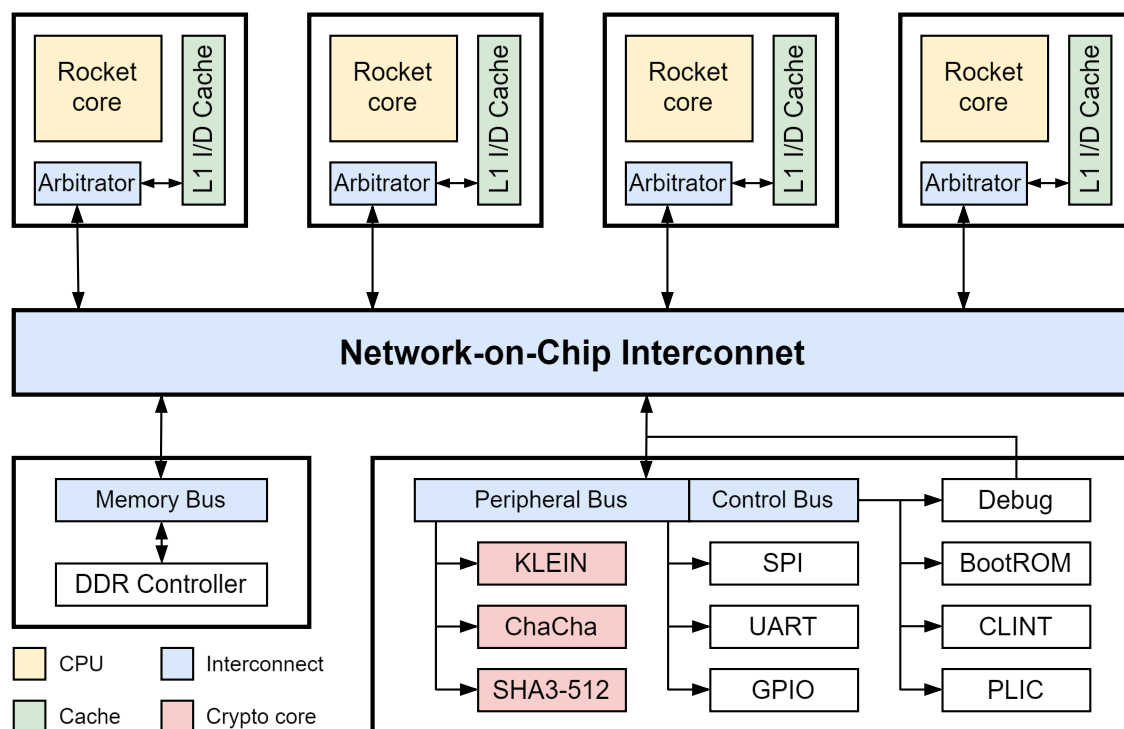
3.2 Tổng quan hệ thống

Hệ thống SoC trong khóa luận được thiết kế với mục tiêu tối ưu hóa cho các ứng dụng xử lý song song, đồng thời hỗ trợ hiệu quả cho các tác vụ mã hóa dữ liệu trong môi trường IoT và edge AI - vốn là những lĩnh vực đòi hỏi hiệu năng cao, tiêu thụ năng lượng thấp và mức độ bảo mật dữ liệu cao.

Kiến trúc tổng thể của hệ thống được thể hiện trong Hình 3.2, bao gồm các thành phần chính như sau:

- Bốn lõi xử lý Rocket (*Rocket Cores*): Đây là các lõi CPU RISC-V 64-bit, được tổ chức theo kiến trúc đa lõi đồng nhất. Mỗi lõi đều có khả năng truy cập bộ nhớ và các thiết bị ngoại vi thông qua mạng NoC, đồng thời có thể thực hiện độc lập hoặc phối hợp trong các bài toán tính toán như nhân ma trận, mã hóa dữ liệu,...
- Các IP ngoại vi tích hợp (*Peripherals*): Bao gồm bộ điều khiển bộ nhớ DDR, các khối gỡ lỗi và điều khiển ngắt, BootROM, UART và các I/O khác, cho phép hệ thống tương tác với thế giới bên ngoài cũng như hỗ trợ lập trình, nạp chương trình và đo kiểm kết quả. Tất cả các ngoại vi đều được ánh xạ vào không gian địa chỉ thông qua cơ chế MMIO.

- Các lõi mật mã hóa phần cứng nhẹ (*Lightweight Cryptographic Cores*): Hệ thống tích hợp nhiều lõi mật mã hóa phần cứng như KLEIN, ChaCha và hàm băm SHA3-512, nhằm tăng cường khả năng bảo mật ngay từ tầng phần cứng. Các lõi này được thiết kế để hoạt động độc lập và được gọi bởi các CPU thông qua giao tiếp MMIO. Việc tích hợp các lõi gia tốc này giúp giảm tải đáng kể cho CPU, đồng thời đảm bảo tốc độ mã hóa phù hợp với yêu cầu thời gian thực của các thiết bị IoT.
- Hệ thống liên kết mạng trên chip (*Network-on-Chip Interconnect*): NoC giữ vai trò là hạ tầng truyền thông nội bộ, kết nối các lõi xử lý, lõi gia tốc và các IP ngoại vi lại với nhau. Hệ thống này sử dụng topology vòng (Ring Topology), kết hợp với thuật toán định tuyến đường đi tối thiểu (Minimal Routing), để triển khai và tối ưu tài nguyên cho các hệ thống có số lõi vừa phải. Nhờ vào NoC, luồng dữ liệu giữa các thành phần được truyền tải hiệu quả, giảm độ trễ và tăng thông lượng tổng thể của hệ thống.

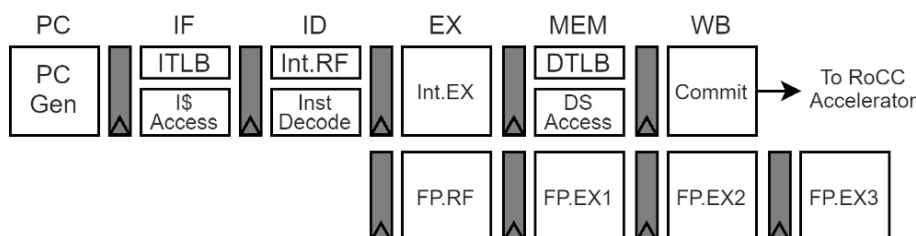


Hình 3.2: Kiến trúc hệ thống đề xuất

Chi tiết các thành phần bên trong hệ thống sẽ được trình bày ở các phần tiếp theo. Với kiến trúc này, hệ thống đề xuất có khả năng mở rộng linh hoạt theo số lượng lõi xử lý hoặc lõi gia tốc chuyên biệt, đồng thời đảm bảo khả năng xử lý tính toán hiệu quả cho các ứng dụng có yêu cầu bảo mật cao.

3.3 Lỗi vi xử lý Rocket

Trong khóa luận này, CPU Rocket [9] được sử dụng với đầy đủ các tập lệnh phổ biến trong kiến trúc RISC-V, bao gồm tập lệnh số nguyên (**I**nteger), nhân và chia (**M**ultiplication/**D**ivision), nguyên tử (**A**tomics), dấu chấm động đơn (**F**loat) và kép (**D**ouble), cũng như tập lệnh nén (**C**ompressed). Ngoài ra, lõi Rocket cũng sử dụng kiến trúc đường ống lệnh (pipelined architecture), cho phép chia nhỏ quá trình thực thi lệnh thành nhiều giai đoạn riêng biệt nhằm tối ưu hiệu suất xử lý. Các giai đoạn này bao gồm: nạp lệnh (Instruction Fetch), giải mã lệnh (Decode), thực thi (Execute), truy cập bộ nhớ (Memory Access) và ghi kết quả (Write Back) như minh họa trong Hình 3.3. Nhờ cấu trúc đường ống này, lõi Rocket có thể thực hiện đồng thời nhiều lệnh ở các giai đoạn khác nhau, tận dụng hiệu quả xử lý song song để nâng cao hiệu năng toàn hệ thống.



Hình 3.3: Kiến trúc đường ống của lõi Rocket

Cụ thể, quá trình xử lý được bắt đầu tại khối nạp lệnh (Fetch Unit), nơi truy xuất các lệnh từ bộ nhớ đệm lệnh (Instruction Cache - I\$), một dạng bộ nhớ đệm L1 có tốc độ cao, lưu trữ những lệnh được sử dụng thường xuyên. Sau khi lệnh được lấy ra, khối giải mã (Decode Unit) sẽ phân tích lệnh bằng cách trích xuất opcode và xác định các toán hạng cùng với các thanh ghi nguồn và đích liên quan.

Tiếp đến, ở giai đoạn thực thi, lệnh được gửi tới các đơn vị thực thi chuyên biệt. Các phép toán số học như cộng, trừ và các phép logic như AND, OR, XOR được xử lý tại đơn vị logic số học (Arithmetic Logic Unit - ALU). Những lệnh yêu cầu phép nhân hoặc chia sẽ được xử lý tại đơn vị nhân/chia (Multiplier/Divider Unit - MUL/DIV). Đối với các lệnh xử lý số thực, đơn vị dấu chấm động (Floating Point Unit - FPU) sẽ đảm nhiệm vai trò tính toán độ chính xác cao.

Tiếp theo là giai đoạn truy cập bộ nhớ. Đối với các lệnh tải hoặc lưu (load/store), bộ tải/lưu (Load/Store Unit - LSU) sẽ truy cập vào bộ nhớ đệm dữ liệu (Data Cache - D\$) hoặc bộ nhớ chính để đọc hoặc ghi dữ liệu. D\$ là bộ nhớ đệm L1 giúp giảm độ trễ truy cập và giảm tải cho hệ thống bus khi nhiều lệnh yêu cầu truy xuất bộ nhớ.

Cơ chế này đặc biệt quan trọng trong các hệ thống đa lõi, nơi các lệnh đồng thời có thể truy cập dữ liệu từ các vùng nhớ riêng biệt hoặc chia sẻ.

Cuối cùng, ở giai đoạn ghi kết quả, các kết quả tính toán được ghi trở lại thành ghi đích để sẵn sàng phục vụ cho các lệnh tiếp theo. Việc ghi kết quả được điều phối cẩn thận để tránh các xung đột hoặc ghi đè không mong muốn. Đối với các lệnh có liên quan đến bộ gia tốc phần cứng như RoCC (Rocket Custom Coprocessor), kết quả có thể được chuyển đến các khối ngoại vi chuyên dụng để tiếp tục xử lý chuyên sâu.

Sự phối hợp chặt chẽ giữa năm giai đoạn này cho phép lõi Rocket đạt được hiệu năng cao trong môi trường xử lý song song, đồng thời duy trì được tính module và khả năng mở rộng cho các hệ thống tích hợp cao như SoC hoặc NoC.

3.4 IP ngoại vi

Các IP ngoại vi trong hệ thống được kế thừa từ các thiết kế của Chipyard và SiFive, bao gồm ba nhóm chính: (i) khối giao tiếp với bộ nhớ ngoài (*DDR Controller*); (ii) các thành phần ngoại vi của bộ xử lý (Uncore) như *BootROM*, *CLINT*, *PLIC* và *Debug*; và (iii) các giao tiếp I/O theo chuẩn như *UART*, *SPI* và *GPIO*.

3.4.1 DDR Controller

Bus bộ nhớ đảm nhiệm việc xử lý các yêu cầu đọc/ghi dữ liệu và truyền lệnh giữa các lõi xử lý và bộ nhớ chính. Để thực hiện chức năng này, hệ thống sử dụng khối *DDR Controller* làm trung gian giao tiếp với module RAM ngoài, đảm bảo luồng dữ liệu diễn ra liên tục, đồng bộ và đạt hiệu suất cao. Bộ điều khiển này còn chịu trách nhiệm điều phối quyền truy cập bộ nhớ từ nhiều lõi, hỗ trợ cơ chế hàng đợi và điều độ truy cập nhằm hạn chế xung đột và giảm độ trễ khi truyền dữ liệu.

3.4.2 Uncore

Các thành phần hỗ trợ hoạt động cho CPU nhưng không thuộc nhóm ngoại vi được gọi là các khối uncore trong hệ sinh thái Chipyard/RocketChip. Các khối này đảm nhận vai trò thiết yếu trong việc khởi động hệ thống, xử lý trình ngắt và hỗ trợ gỡ lỗi phần mềm, góp phần đảm bảo quá trình vận hành của hệ thống được ổn định và chính xác ngay từ khi cấp nguồn.

Một trong những thành phần uncore quan trọng là *BootROM* (Boot Read-Only Memory), nơi lưu trữ firmware phục vụ cho quá trình khởi động ban đầu của hệ

thống. Bộ firmware này đóng vai trò như một trình nạp khởi động (bootloader), chịu trách nhiệm thiết lập môi trường phần cứng ban đầu, cấu hình các thanh ghi hệ thống và chuyển quyền điều khiển cho chương trình người dùng ở các mức cao hơn. Trong hệ thống đề xuất, firmware ở *BootROM* sẽ được thiết kế giống với quy trình phát triển phần mềm đã trình bày, đảm bảo tính tương thích và khả năng khởi động chính xác trên các cấu hình lõi đa dạng.

CLINT (Core-Local Interruptor) là khối được SiFive định nghĩa để xử lý các ngắt nội bộ (local interrupts) theo đặc tả trong tài liệu RISC-V Privileged Architecture Specification v1.10 [32]. Khối này hỗ trợ hai loại ngắt chính là ngắt phần mềm (software interrupt) và ngắt bộ định thời (timer interrupt). Ngoài ra, các ngắt nội bộ khác cũng có thể mở rộng tùy theo hệ thống. Trong hệ thống này, *CLINT* đóng vai trò quản lý các thanh ghi điều khiển và thanh ghi trạng thái liên quan đến các ngắt nội bộ, được ánh xạ vào bộ nhớ và truy cập thông qua phần mềm.

Một trong những thanh ghi quan trọng là MSIP (Machine Software Interrupt Pending). Thanh ghi này có chức năng kích hoạt các hart trong hệ thống đa lõi thông qua cơ chế ngắt phần mềm. Cụ thể, mỗi hart¹ sẽ được đánh thức khi bit tương ứng trong thanh ghi MSIP được thiết lập. Bảng 3.1 thể hiện không gian địa chỉ bộ nhớ của các thanh ghi MSIP tương ứng với từng hart khác nhau, trong đó địa chỉ cơ sở của MSIP là 0x02000000 (ứng với hart 0). Địa chỉ của các hart tiếp theo được tính theo công thức: $BASE_ADDRESS + HART_ID \times 4$.

Bảng 3.1: Sơ đồ các thanh ghi MSIP của *CLINT*

Địa chỉ	Kích thước	Thuộc tính	Mô tả
0x02000000	4 Byte	Read/Write	msip cho hart 0
0x02000004	4 Byte	Read/Write	msip cho hart 1
0x02000008	4 Byte	Read/Write	msip cho hart 2
0x0200000C	4 Byte	Read/Write	msip cho hart 3

Mặc dù mỗi thanh ghi MSIP có độ rộng 32 bit, chỉ có bit 0 được sử dụng để đại diện cho trạng thái pending của ngắt phần mềm. Khi bit này được thiết lập, nó sẽ đặt bit tương ứng trong thanh ghi *mip* (Machine Interrupt Pending) của mỗi lõi xử lý. Nếu đồng thời các bit MSIE (Machine Software Interrupt Enable) trong thanh ghi *mie* (Machine Interrupt Enable) và bit MIE trong thanh ghi *mstatus* (Machine Status) được bật, thì hart sẽ nhảy vào trình xử lý ngắt để thực thi ngắt phần mềm

¹Trong RISC-V, "hart" là viết tắt của "hardware thread", chỉ một luồng xử lý phần cứng. Mỗi hart có một đơn vị lấy lệnh riêng và một ID duy nhất.

tương ứng. Hình 3.4, Hình 3.5 và Hình 3.6 lần lượt minh họa vị trí chuẩn của các bit trong các thanh ghi *mip*, *mie* và *mstatus* của kiến trúc RISC-V. Đáng chú ý, các bit được sử dụng trong cơ chế điều khiển liên quan đến ngắt phần mềm đều nằm tại vị trí bit thứ 3 trong mỗi thanh ghi tương ứng.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	LCOFIP	0	MEIP	0	SEIP	0	MTIP	0	STIP	0	MSIP	0	SSIP	0	0

Hình 3.4: Phần chuẩn (các bit từ 15 đến 0) của thanh ghi *mip*

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	LCOFIE	0	MEIE	0	SEIE	0	MTIE	0	STIE	0	MSIE	0	SSIE	0	0

Hình 3.5: Phần chuẩn (các bit từ 15 đến 0) của thanh ghi *mie*

31	30								23	22	21	20	19	18	17	16
SD	WPRI								TSR	TW	TVM	MXR	SUM	MPRV	XS[1]	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
XS[0]	FS[1:0]	MPP[1:0]	VS[1:0]	SPP	MPIE	UBE	SPIE	WPRI	MIE	WPRI	SIE	WPRI				

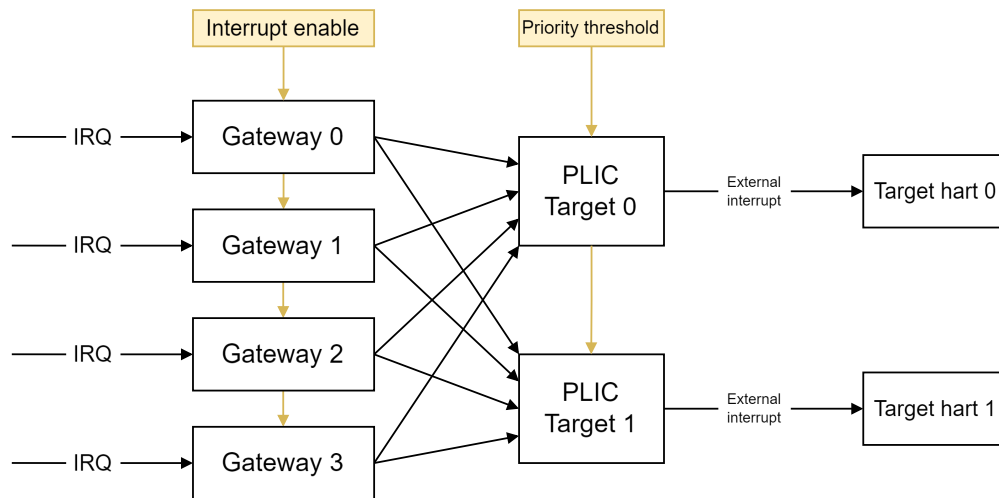
Hình 3.6: Phần chuẩn (các bit từ 31 đến 0) của thanh ghi *mstatus*

PLIC (Platform-Level Interrupt Controller) là khối phần cứng chịu trách nhiệm quản lý và định tuyến các ngắt ngoại vi trong hệ thống RISC-V đa lõi. Mỗi nguồn ngắt được gán một mức ưu tiên (thường từ 1 đến 7, với 0 là không có ngắt) và có thể được bật hoặc tắt thông qua các thanh ghi điều khiển ánh xạ bộ nhớ. Khi một ngắt được kích hoạt, *PLIC* sẽ so sánh mức ưu tiên với mức ngưỡng ưu tiên *priority threshold* hiện tại, sắp xếp và gửi ngắt có ưu tiên cao nhất đến lõi xử lý tương ứng.

Quá trình xử lý ngắt tuân theo cơ chế *claim-complete*, trong đó lõi xử lý sẽ truy vấn thông tin ngắt thông qua thao tác *claim*, thực hiện trình phục vụ ngắt, sau đó ghi lại kết quả *complete* để thông báo đã xử lý xong. Thiết kế đơn giản, dễ mở rộng và hỗ trợ đa nguồn (lên đến 51 nguồn) - đa đích giúp *PLIC* phù hợp với các hệ thống SoC hiện đại có nhiều thiết bị ngoại vi.

Hình 3.7 mô tả kiến trúc hoạt động của *PLIC* với bốn nguồn ngắt ngoại vi (*IRQ*) đi qua các khối *Gateway*, được điều khiển bởi cấu hình *interrupt enable*. Sau đó, các tín hiệu ngắt được định tuyến đến các lõi (*PLIC Target 0*, *PLIC Target 1*), nơi *priority threshold* được so sánh để quyết định có xử lý ngắt hay không. Cuối cùng,

tín hiệu *external interrupt* được gửi đến các *target hart* tương ứng để kích hoạt quá trình xử lý ngắt trong phần mềm.



Hình 3.7: Kiến trúc hoạt động của *PLIC* trong hệ thống đa lõi

Khối *Debug* cung cấp cơ chế hỗ trợ gỡ lỗi phần mềm và phần cứng trong hệ thống RISC-V. Thông qua giao diện JTAG, khối này cho phép truy cập vào các thanh ghi của lõi xử lý (GPR, CSR, FPR), cũng như bộ nhớ hệ thống được ánh xạ. Cấu trúc debug bao gồm ba phần: phần mềm gỡ lỗi trên máy chủ (như GDB), thiết bị phần cứng trung gian (J-Link, TRACE32) và khối *Debug* bên trong chip. Ngoài khả năng dừng, tiếp tục hoặc đặt breakpoint, hệ thống còn hỗ trợ các lệnh trừu tượng (abstract commands) để truy xuất nhanh trạng thái hệ thống mà không ảnh hưởng đến luồng thực thi chính.

3.4.3 Giao tiếp I/O

Đây là các khối ngoại vi cơ bản được tích hợp nhằm đa dạng hóa khả năng giao tiếp I/O của hệ thống và hỗ trợ triển khai các ứng dụng thực tế. Các giao tiếp bao gồm: *UART* (Universal Asynchronous Receiver-Transmitter), *SPI* (Serial Peripheral Interface) và *GPIO* (General Purpose Input/Output).

Trong đó, *UART* đóng vai trò cầu nối truyền thông nối tiếp không đồng bộ giữa hệ thống và máy tính. Khối này được cấu hình với tốc độ truyền 115200 baud, cho phép in thông tin hệ thống lên terminal để phục vụ mục đích kiểm tra, gỡ lỗi hoặc giao tiếp cơ bản với người dùng trong môi trường không đồ họa.

SPI là chuẩn giao tiếp nối tiếp đồng bộ tốc độ cao, được sử dụng trong hệ thống nhằm kết nối với thẻ nhớ SD Card. Thông qua *SPI*, hệ thống có thể tải chương trình

ứng dụng từ bộ nhớ ngoài vào trong quá trình khởi động hoặc thực thi, đặc biệt hữu ích trong các hệ thống nhúng.

Cuối cùng, *GPIO* là tập hợp các chân I/O mục đích chung, cho phép người thiết kế có thể cấu hình để giao tiếp với nhiều loại thiết bị ngoại vi như cảm biến, led, relay hoặc các module mở rộng khác. Việc tích hợp *GPIO* giúp hệ thống mở rộng khả năng tương tác với môi trường và đáp ứng linh hoạt các yêu cầu truyền thông đa dạng trong các ứng dụng *IoT* hoặc điều khiển nhúng.

3.5 Lỗi mật mã hóa nhẹ

Phần này trình bày các thuật toán mật mã hóa nhẹ được tích hợp trong hệ thống đề xuất. Đây là những thuật toán được thiết kế tối ưu cho các thiết bị và ứng dụng IoT, nơi tài nguyên phần cứng hạn chế và yêu cầu tiêu thụ năng lượng thấp là yếu tố quan trọng. Các thuật toán được lựa chọn đều có chi phí phần cứng thấp, tốc độ xử lý cao và mức độ bảo mật phù hợp với môi trường nhúng.

3.5.1 Mã hóa khối KLEIN

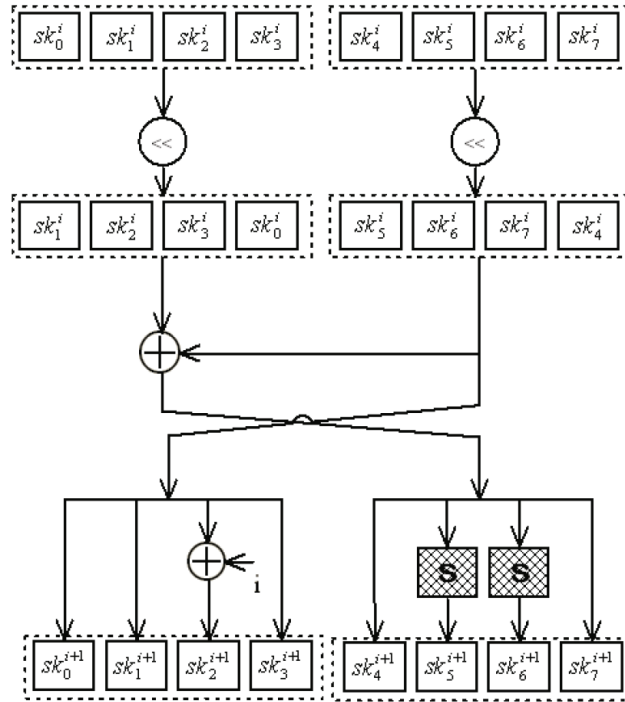
KLEIN là một thuật toán mã hóa khối nhẹ (lightweight block cipher) được thiết kế nhằm cân bằng giữa tính bảo mật và khả năng triển khai hiệu quả trên phần cứng hạn chế [12]. Thuật toán này sử dụng cấu trúc mạng thay thế-hoán vị (Substitution-Permutation Network - SPN) và có ba biến thể chính: KLEIN-64, KLEIN-80 và KLEIN-96, tương ứng với độ dài khóa 64, 80 và 96-bit, cùng số vòng lặp lần lượt là 12, 16 và 20. Trong khuôn khổ khóa luận này, biến thể KLEIN-64 được lựa chọn do có cấu trúc đơn giản và chi phí phần cứng thấp, với độ dài bản rõ (plaintext), khóa (key) và bản mã (ciphertext) đều là 64-bit.

Thuật toán 1 Quy trình mã hóa của KLEIN-64

- 1: $sk^1 \leftarrow \text{KEY}$
 - 2: $\text{STATE} \leftarrow \text{PLAINTEXT}$
 - 3: **for** $i = 1$ to N_R **do**
 - 4: AddRoundKey(STATE , sk^i)
 - 5: SubNibbles(STATE)
 - 6: RotateNibbles(STATE)
 - 7: MixNibbles(STATE)
 - 8: $sk^{i+1} \leftarrow \text{KeySchedule}(sk^i, i)$
 - 9: $\text{CIPHERTEXT} \leftarrow \text{AddRoundKey}(\text{STATE}, sk^{N_R+1})$
-

Quá trình mã hóa trong KLEIN-64 được thực hiện qua 12 vòng lặp, mỗi vòng bao gồm bốn bước xử lý chính như mô tả trong Thuật toán 1: (1) *AddRoundKey*: thực hiện phép XOR giữa bản rõ ở trạng thái hiện tại và khóa vòng; (2) *SubNibbles*: thay thế từng nibble 4-bit thông qua S-box cố định; (3) *RotateNibbles*: dịch vòng 2-byte; (4) *MixNibbles*: thực hiện biến đổi tương tự như bước MixColumn trong thuật toán AES [5]. Sau mỗi vòng, một khóa con mới cũng được tạo ra thông qua thuật toán *KeySchedule*, đảm bảo thay đổi khóa liên tục giữa các vòng.

Thuật toán *KeySchedule* tạo khóa con mới từ khóa con hiện tại bằng các thuật toán hoán vị vòng, XOR và áp dụng S-box như trong Hình 3.8. Cơ chế này giúp tạo ra chuỗi khóa con đủ ngẫu nhiên, tránh các mẫu lặp lại giữa các vòng.



Hình 3.8: Thuật toán *KeySchedule* với độ dài khóa 64-bit

3.5.2 Mã hóa dòng ChaCha

ChaCha là một thuật toán mã hóa dòng (stream cipher) được phát triển dựa trên Salsa20 [3], sử dụng cấu trúc toán học gọi là Add-Rotate-XOR (ARX) nhằm đảm bảo tính bảo mật đồng thời tối ưu cho việc triển khai trên phần cứng và phần mềm. Thuật toán nhận vào ba tham số chính: *key* 256-bit, *nonce* (giá trị khởi tạo) 64-bit và *counter* 64-bit, kết hợp cùng với bốn word 32-bit hằng số có nội dung là “*expand 32-byte k*”. Tất cả các thành phần này được sắp xếp thành một ma trận 4×4 , tạo nên trạng thái khởi tạo 512-bit ban đầu như minh họa trong Hình 3.9.

"expa"	"nd 3"	"2-by"	"te k"
Key	Key	Key	Key
Key	Key	Key	Key
Counter	Counter	Nonce	Nonce

Hình 3.9: Trạng thái khởi tạo của bộ mã dòng *ChaCha*

Mỗi vòng mã hóa trong *ChaCha* sẽ thực hiện nhiều lần phép toán gọi là *Quarter-Round*, là thành phần lõi của thuật toán. Cụ thể, một vòng đầy đủ sẽ bao gồm bốn phép theo hàng và bốn phép theo cột hoặc đường chéo. Hàm *Quarter-Round* này bao gồm các phép cộng module, dịch vòng trái và XOR, hoạt động trên bốn word 32-bit (gọi là a, b, c, d) như sau:

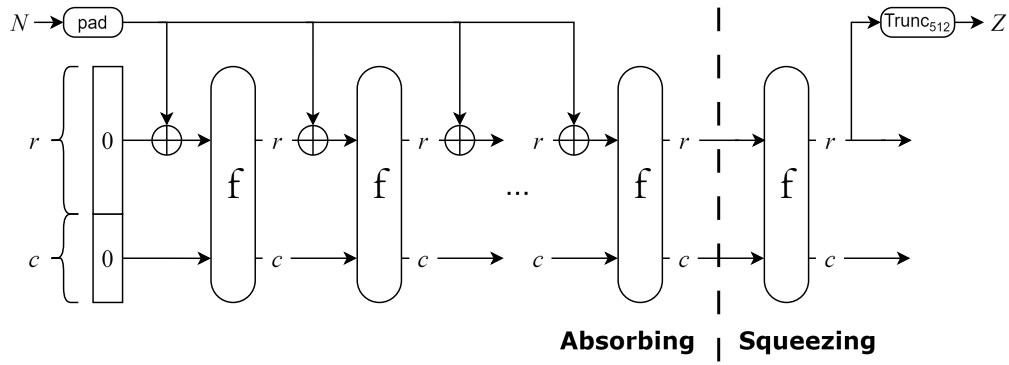
1. $a = a + b; \quad d = d \oplus a; \quad d = \text{ROTL}(d, 16)$
2. $c = c + d; \quad b = b \oplus c; \quad b = \text{ROTL}(b, 12)$
3. $a = a + b; \quad d = d \oplus a; \quad d = \text{ROTL}(d, 8)$
4. $c = c + d; \quad b = b \oplus c; \quad b = \text{ROTL}(b, 7)$

Sau khi hoàn tất tất cả các vòng lặp (thường là 20 vòng), trạng thái đầu ra sẽ được cộng với trạng thái ban đầu để tạo ra *keystream*. Cuối cùng, *keystream* được XOR với bản rõ để tạo ra bản mã. Do không có bước thay thế S-box như các thuật toán khối truyền thống, *ChaCha* rất thích hợp để triển khai trên vi xử lý nhúng và thiết bị IoT nhờ vào cấu trúc số học đơn giản nhưng hiệu quả về mặt bảo mật và hiệu suất.

3.5.3 Hàm băm SHA3-512

SHA3-512 là một thuật toán băm mật mã thuộc họ tiêu chuẩn Secure Hash Algorithm 3 (SHA-3) [30], được phát triển dựa trên cấu trúc Sponge Construction và sử dụng hàm hoán vị Keccak-f[1600]. Cấu trúc này cho phép *SHA3-512* đạt được tính bảo mật cao và chống lại được nhiều kiểu tấn công hiện đại, nhờ vào hai giai đoạn tách biệt rõ ràng là hấp thụ (*absorbing*) và vắt (*squeezing*), như trong Hình 3.10.

Quá trình băm bắt đầu bằng việc bổ sung phần đệm (*padding*) cho thông điệp đầu vào theo quy tắc $\text{pad}10*1$, sau đó chia nhỏ thành các khối 576-bit - còn gọi là *rate* (r). Các khối này lần lượt được đưa vào trạng thái 1600-bit thông qua phép XOR và hàm Keccak-f. Trạng thái này bao gồm hai phần: 576-bit dùng cho *rate* và 1024-bit còn lại dùng làm *capacity* (c).



Hình 3.10: Cấu trúc sponge của thuật toán *SHA3-512*

Sau khi toàn bộ dữ liệu được hấp thụ, thuật toán chuyển sang giai đoạn *squeezing*, trong đó hàm băm 512-bit được tạo ra bằng cách trích xuất trực tiếp phần đầu của trạng thái - tương ứng với phần r đầu tiên. Nhờ việc $r > 512$, hàm băm có thể được tạo ra chỉ sau một lần vắt duy nhất, tối ưu hiệu suất và giảm độ trễ.

Mặc dù *SHA3-512* không hoàn toàn thuộc nhóm các thuật toán mã hóa nhẹ, thuật toán này vẫn được lựa chọn và tích hợp trong khóa luận nhằm mở rộng tính đa dạng về loại hình và họ thuật toán mật mã trong hệ thống SoC. Việc thêm *SHA3-512* góp phần bổ sung chức năng băm mật mã, bên cạnh mã hóa khối và dòng.

3.6 Liên kết mạng trên chip

Để khắc phục các hạn chế về hiệu năng và khả năng mở rộng của kiến trúc bus truyền thống trong các hệ thống xử lý đa lõi, kiến trúc SoC trong khóa luận này đã thay thế bus truyền thống bằng Network-on-Chip. Việc áp dụng NoC mang lại nhiều lợi ích như tăng băng thông, giảm độ trễ và cải thiện khả năng mở rộng khi số lượng lõi xử lý tăng.

Một hệ thống NoC hiệu quả cần được thiết kế dựa trên ba yếu tố chính như đã nhắc ở phần 2.1.4: (i) kiến trúc mạng; (ii) thuật toán định tuyến; và (iii) cơ chế điều khiển luồng. Do phạm vi lý thuyết về NoC rất rộng, phần trình bày sau sẽ chỉ tập trung vào các khái niệm và kỹ thuật được áp dụng cụ thể trong khuôn khổ đề tài.

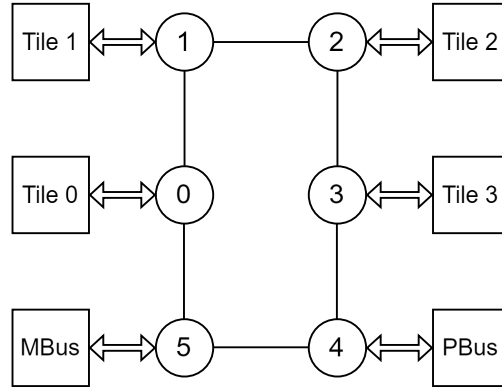
3.6.1 Kiến trúc mạng

Trong thiết kế NoC, topology là yếu tố quyết định cách bố trí vật lý và kết nối giữa các nút (node) trong mạng. Topology có ảnh hưởng trực tiếp đến hiệu suất toàn hệ thống, bao gồm độ trễ, băng thông, khả năng phân phối tải và mức tiêu thụ năng lượng. Cụ thể, cấu trúc mạng sẽ quyết định số bước nhảy (hop count), tải tối đa trên

kênh truyền (maximum channel load) và mức độ đa dạng đường đi của các gói tin (path diversity), từ đó tác động mạnh hiệu quả truyền thông. Độ phức tạp của một topology thường được đánh giá qua số liên kết vật lý tại mỗi node (node degree) và cách sắp xếp các kết nối trên mặt phẳng chip.

Các cấu trúc topology phổ biến trong NoC được chia thành ba nhóm chính: mạng trực tiếp (direct network), mạng gián tiếp (indirect network) và mạng không đều (irregular network). Mạng trực tiếp kết nối các node đến các router lân cận - ví dụ như mesh, torus, hoặc ring - với ưu điểm dễ triển khai và định tuyến đơn giản. Ngược lại, mạng gián tiếp như fat-tree hoặc butterfly sử dụng các tầng chuyển mạch trung gian, phù hợp với hệ thống yêu cầu độ sâu lớn và hiệu quả định tuyến cao. Mạng không đều là sự pha trộn linh hoạt giữa các cấu trúc trên, giúp tối ưu hóa băng thông và giảm độ trễ trong các hệ thống không đồng nhất.

Trong hệ thống được đề xuất, NoC sử dụng topology vòng (ring topology), thuộc loại mạng trực tiếp. Đây là cấu trúc đơn giản, tối ưu về mặt tài nguyên nhờ giảm thiểu số kết nối vật lý (channel) giữa các router. Mỗi node được gắn với một router riêng, và toàn bộ các router được kết nối nối tiếp với nhau thành một vòng khép kín. Cấu trúc này đảm bảo tính kết nối đầy đủ trong hệ thống, đồng thời giúp tiết kiệm diện tích chip và điện năng tiêu thụ.



Hình 3.11: Topology trong hệ thống NoC đề xuất

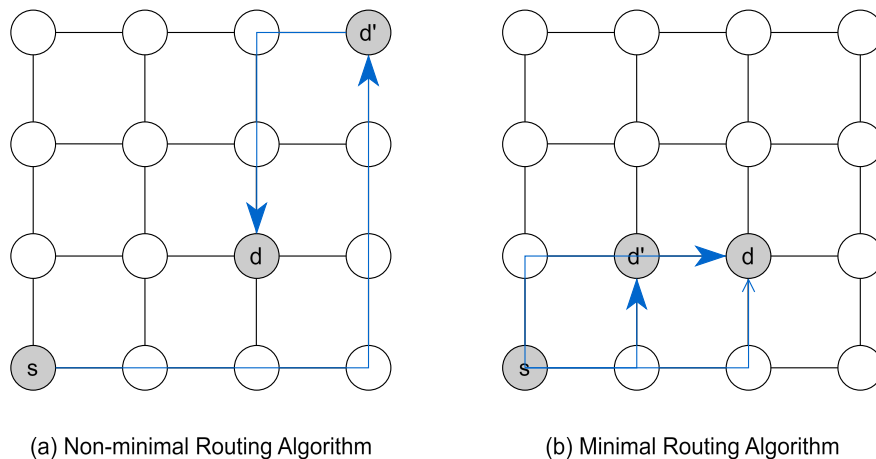
Trong Hình 3.11, liên kết vật lý giữa các router được biểu diễn bằng các đường thẳng, trong khi các mũi tên hai chiều thể hiện kết nối chuẩn TileLink [29]. Hệ thống hỗ trợ hai chuẩn TileLink là TileLink Uncached Lightweight (TL-UL) và TileLink Cached (TL-C). Cụ thể, TL-UL được sử dụng cho bus ngoại vi (*PBus*) với hai kênh A và D, trong khi TL-C được áp dụng cho các liên kết còn lại với đầy đủ năm kênh từ A đến E. Bốn *Tile* trong hệ thống được kết nối trực tiếp với bốn router tương ứng, trong khi router 4 kết nối với *PBus* và router 5 kết nối với bus bộ nhớ chính (*MBus*).

3.6.2 Thuật toán định tuyến

Sau khi xác định topology mạng, bước tiếp theo là lựa chọn thuật toán định tuyến - yếu tố quyết định cách thức một thông điệp di chuyển qua các node để đến đích. Mục tiêu của định tuyến là phân bổ lưu lượng hợp lý nhằm tránh điểm nóng, giảm xung đột và tối ưu độ trễ. Việc lựa chọn thuật toán phụ thuộc vào các tiêu chí như độ phức tạp phần cứng, khả năng mở rộng và mức tiêu thụ năng lượng.

Các thuật toán định tuyến thường được phân thành hai nhóm chính: xác định (deterministic), không nhận biết (oblivious) hoặc thích ứng (adaptive) và tối thiểu (minimal) hoặc không tối thiểu (non-minimal). Trong khuôn khổ đề tài, trọng tâm sẽ đặt vào định tuyến tối thiểu và không tối thiểu.

Định tuyến tối thiểu đảm bảo mỗi gói tin đi theo đường ngắn nhất từ nguồn đến đích. Ví dụ, trong mạng mesh 2D với node nguồn tại $(0,0)$ và đích tại (i,j) , số bước nhảy tối thiểu là $|i| + |j|$. Ngược lại, định tuyến không tối thiểu cho phép sử dụng các đường đi dài hơn nhằm phân tán lưu lượng và tránh tắc nghẽn, song có thể làm tăng độ trễ và mức tiêu thụ năng lượng.



Hình 3.12: Ví dụ về định tuyến tối thiểu và không tối thiểu

Hình 3.12 minh họa sự khác biệt: thuật toán *non-minimal* chọn ngẫu nhiên một điểm đích trung gian d' trước khi đến đích thực, làm tăng số bước nhảy từ 3 lên 9. Trong khi đó, thuật toán *minimal* giới hạn d' trong góc phần tư tối thiểu, giúp duy trì đường đi ngắn nhất.

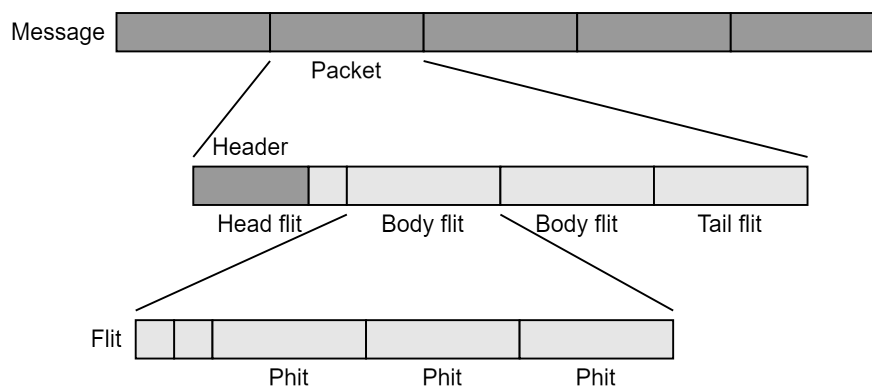
Trong hệ thống này, thuật toán định tuyến áp dụng là tối thiểu và tĩnh. Các đường đi được xác định trước khi truyền, không phụ thuộc vào trạng thái mạng. Cách tiếp cận này giúp đơn giản hóa phần cứng router, tránh hiện tượng deadlock, đồng thời đảm bảo dữ liệu di chuyển theo lộ trình tối ưu nhất về độ trễ.

3.6.3 Cơ chế điều khiển luồng

Cơ chế điều khiển luồng của hệ thống được triển khai theo phương thức credit-based kết hợp với kỹ thuật chuyển mạch wormhole. Phương pháp này giúp tối ưu hóa tài nguyên bộ đệm tại các router và kiểm soát lưu lượng dữ liệu trong mạng. Những khái niệm chi tiết về cấu trúc dữ liệu (flit, phit), chuyển mạch wormhole và điều khiển luồng dựa trên credit sẽ được trình bày trong phần các phần bên dưới.

a) Chiến Lược Chuyển Mạch (Switching Strategies)

Chiến lược chuyển mạch xác định cách dữ liệu di chuyển giữa các router trong mạng NoC. Trong các hệ thống NoC, dữ liệu được tổ chức theo bốn cấp: message, packet, flit và phit. Một message thường bao gồm nhiều packets, mỗi packet được chia nhỏ thành nhiều flits và mỗi flit lại được cấu thành từ một hoặc nhiều phits. Flit (flow control unit) là đơn vị cơ bản để thực hiện kiểm soát luồng, đồng thời đóng vai trò là đơn vị đồng bộ giữa các router. Trong khi đó, phit (physical unit) là phần dữ liệu nhỏ nhất được truyền trên liên kết vật lý trong một chu kỳ xung nhịp, với kích thước thường tương ứng với độ rộng bit của đường truyền. Hình 3.13 minh họa cấu trúc phân cấp giữa các thành phần này.



Hình 3.13: Cấu trúc của messages, packets, flits và phits

Các kiến trúc NoC khác nhau có thể lựa chọn kích thước phit, flit, packet và message khác nhau, tùy thuộc vào yêu cầu về hiệu suất, chi phí phần cứng và mức tiêu thụ năng lượng. Hai chiến lược chuyển mạch thường thấy trong NoC là chuyển mạch mạch (circuit switching) và chuyển mạch gói (packet switching).

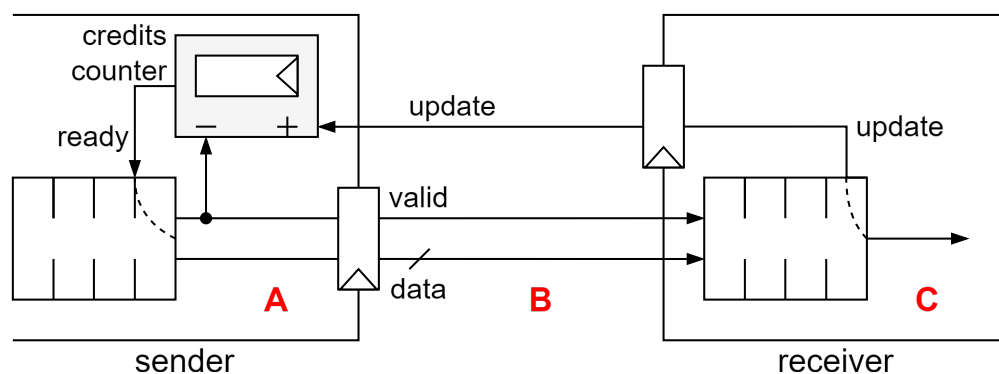
Trong chuyển mạch mạch, một đường dẫn vật lý được thiết lập sẵn giữa router nguồn và đích trước khi truyền dữ liệu, kéo theo một khoảng trễ thiết lập ban đầu (setup latency), nhưng đổi lại đảm bảo băng thông cố định cho toàn bộ phiên truyền. Ngược lại, trong chuyển mạch gói, các packet được truyền độc lập mà không cần thiết lập trước đường dẫn. Các packet có thể đi theo các tuyến đường khác nhau với độ

trễ biến thiên, phụ thuộc vào tình trạng tắc nghẽn trong mạng. Tuy nhiên, khi nhiều packet từ các nguồn khác nhau đồng thời cạnh tranh cùng một liên kết tại router, chỉ một packet được ưu tiên đi tiếp, trong khi các packet còn lại phải chờ, làm giảm khả năng đảm bảo chất lượng dịch vụ so với chuyển mạch mạch.

Wormhole switching là một kỹ thuật phổ biến thuộc chuyển mạch gói, trong đó yêu cầu bộ đệm tại router chỉ cần đủ cho một flit thay vì toàn bộ packet. Mỗi flit sẽ được chuyển tiếp tới router tiếp theo nếu còn đủ không gian bộ đệm. Tuy nhiên, khi router kế tiếp không còn chỗ trống, phần còn lại của packet sẽ bị giữ lại, dẫn đến việc chiếm dụng liên kết kéo dài và nguy cơ tắc nghẽn cao hơn so với các kỹ thuật khác. Chuyển mạch wormhole cũng dễ gặp hiện tượng deadlock hơn, do các phụ thuộc tài nguyên hình thành khi nhiều packet chia sẻ cùng một liên kết. Mặc dù vậy, phần lớn các kiến trúc NoC hiện nay vẫn ưu tiên sử dụng wormhole switching nhờ ưu điểm tiết kiệm tài nguyên phần cứng.

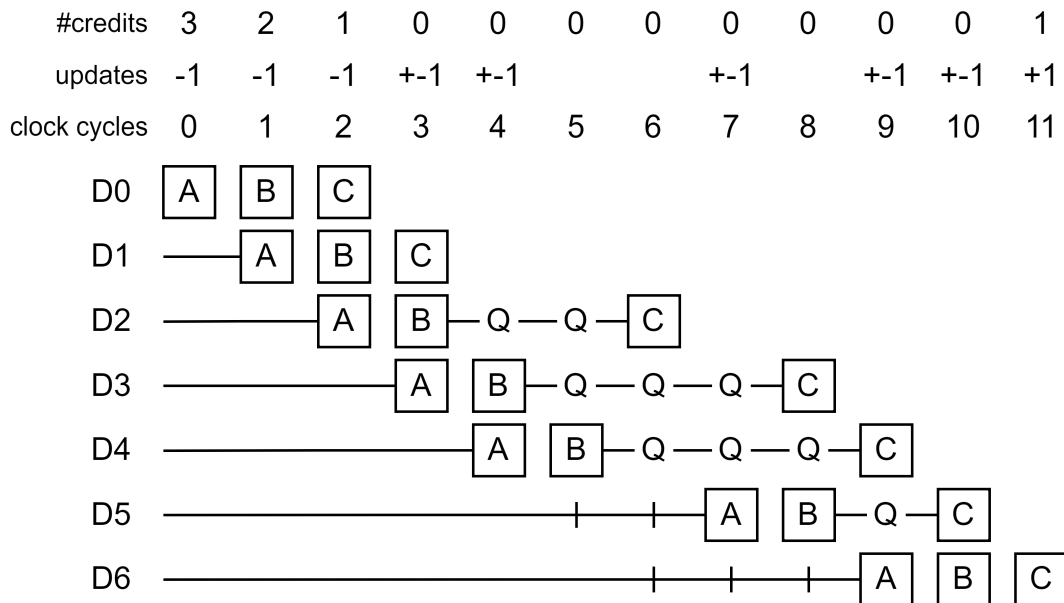
b) Credit-Based Flow Control

Khi bộ đếm (*counter*) được gắn với bộ phát (*sender*), cơ chế điều khiển luồng phát sinh được gọi là điều khiển luồng dựa trên tín dụng (*credit-based flow control*), như minh họa trong Hình 3.14. Cơ chế này cung cấp cho bộ phát toàn bộ thông tin cần thiết để bắt đầu, tạm dừng hoặc tiếp tục quá trình truyền dữ liệu [16], [6]. Trong hệ thống, bộ phát duy trì một bộ đếm tín dụng (*credit counter*) để theo dõi số lượng khe đệm (*buffer slots*) còn trống tại bộ thu (*receiver*). Giá trị này được gọi là tín dụng (*credits*) và được lưu trữ ở phía bộ phát. Khi số credit lớn hơn 0, bộ phát được phép gửi một flit dữ liệu, đồng thời tiêu hao một credit tương ứng với một khe đệm trống tại bộ thu. Sau mỗi lần truyền, bộ đếm tín dụng giảm đi một đơn vị để phản ánh việc một khe đệm đã bị chiếm dụng. Khi một flit dữ liệu được xử lý xong tại đầu ra của bộ thu - tức là đã rời khỏi bộ đệm đầu vào - tín hiệu cập nhật tín dụng (*credit update signal*) sẽ được gửi ngược về bộ phát để tăng giá trị credit.



Hình 3.14: Sơ đồ tổ chức của cơ chế điều khiển luồng dựa trên tín dụng

Một ví dụ cụ thể về hoạt động của cơ chế điều khiển luồng dựa trên tín dụng được thể hiện trong Hình 3.15. Ban đầu, bộ đếm tín dụng tại bộ phát được khởi tạo với giá trị 3, tương ứng với khả năng sử dụng tối đa 3 khe đệm tại bộ thu. Trong quá trình truyền, miễn là giá trị credit còn lớn hơn 0, bộ phát tiếp tục gửi các flit dữ liệu về phía bộ thu. Mỗi khi bộ thu xử lý xong một flit dữ liệu, nó phát tín hiệu cập nhật tín dụng trở lại bộ phát để thông báo đã giải phóng một khe đệm. Tuy nhiên, tín hiệu này luôn đến bộ phát với một chu kỳ trễ.



Hình 3.15: Minh họa quá trình truyền dữ liệu giữa bộ phát và bộ thu

Một điểm đáng lưu ý là ngay trong chu kỳ mà tín hiệu cập nhật tín dụng tới bộ phát, credit này có thể được tái sử dụng lập tức để gửi dữ liệu mới. Trạng thái này được thể hiện trong hình bằng các chu kỳ ký hiệu $+1$, nơi giá trị credit không thay đổi: nó vừa được cộng thêm nhờ tín hiệu cập nhật, đồng thời lập tức bị trừ đi do việc gửi dữ liệu tiếp theo.

Ngoài ra, nếu tốc độ xử lý của bộ thu hoặc thành phần sink (C) chậm hơn tốc độ truyền từ bộ phát, các flit dữ liệu sẽ tạm thời lưu lại trong hàng đợi (queue, ký hiệu Q) tại bộ thu. Sự xuất hiện của Q trong hình cho thấy dữ liệu đã được nhận vào bộ đệm nhưng chưa kịp xử lý hoặc đẩy tiếp tới sink. Mặc dù vậy, pipeline truyền dữ liệu vẫn tiếp tục vận hành: bộ phát vẫn có thể gửi thêm dữ liệu miễn là còn credit khả dụng, bất chấp việc các flit dữ liệu trước đó vẫn đang xếp hàng chờ xử lý trong bộ đệm. Cơ chế này đảm bảo luồng dữ liệu diễn ra liên tục mà không gây mất mát hay bỏ sót, vì mỗi flit dữ liệu chỉ được truyền đi sau khi đã bảo đảm có đủ credit, tức là đã “đặt chỗ trước” trong bộ đệm của bộ thu.

3.7 Thuật toán nhân ma trận

Nhân ma trận là một phép toán cơ bản, đóng vai trò quan trọng trong nhiều lĩnh vực tính toán hiệu năng cao như xử lý tín hiệu số, thị giác máy tính và học máy. Trong khuôn khổ đề tài, phép nhân hai ma trận vuông kích thước $p \times p$ được lựa chọn làm bài toán thử nghiệm để đánh giá khả năng thực thi song song của các lõi xử lý RISC-V trên hệ thống SoC đa lõi.

Thuật toán 2 Matrix Multiplication $\rightarrow MatMul(C, A, B)$

Đầu vào: hai ma trận A và B kích thước $p \times p$

Đầu ra: ma trận C là kết quả với kích thước phù hợp

```
1: for  $i = 0$  to  $p - 1$  do
2:   for  $j = 0$  to  $p - 1$  do
3:      $sum \leftarrow 0$ 
4:     for  $k = 0$  to  $p - 1$  do
5:        $sum \leftarrow sum + A_{ik} \times B_{kj}$ 
6:      $C_{ij} \leftarrow sum$ 
```

Thuật toán 2 mô tả cách nhân hai ma trận A và B để tạo ra ma trận kết quả C cùng kích thước. Thuật toán sử dụng ba vòng lặp lồng nhau (iterative) để thực hiện phép nhân tích lũy (multiply-accumulate). Mỗi phần tử C_{ij} được tính bằng tổng $\sum_{k=0}^{p-1} A_{ik} \times B_{kj}$. Đây là thuật toán cơ bản với độ phức tạp tính toán $O(p^3)$.

Để khai thác khả năng song song của hệ thống đa lõi, đề tài triển khai phiên bản song song của thuật toán nhân ma trận, phân chia khối lượng công việc cho nhiều lõi xử lý RISC-V hoạt động đồng thời trong hệ thống NoC.

Chiến lược song song hóa áp dụng là phân chia theo hàng (*row-wise partitioning*), như trình bày trong thuật toán 3. Mỗi hart đảm nhiệm tính toán một tập con các hàng của ma trận kết quả C . Cụ thể, trong hệ thống có bốn hart, mỗi hart có chỉ số id ($0 \leq id < 4$) sẽ xử lý các hàng từ chỉ số $\frac{p}{4} \times id$ đến $\frac{p}{4} \times (id + 1) - 1$. Phân chia này giúp đảm bảo phân bổ đều khối lượng tính toán và tối ưu hiệu suất xử lý song song.

Trong quá trình thực thi, mỗi hart chỉ cần truy cập phần dữ liệu tương ứng của ma trận A và toàn bộ ma trận B . Vì ma trận B được sử dụng chung cho mọi phép tính C_{ij} , các phần tử của B có thể được ánh xạ vào bộ nhớ chung hoặc sao chép vào vùng nhớ cục bộ của từng hart để tăng tốc độ truy xuất. Sau khi tính toán xong, các hart sẽ ghi kết quả vào các vùng bộ nhớ riêng biệt tương ứng của ma trận C .

Cách tiếp cận này giúp rút ngắn thời gian tính toán gần như tuyến tính theo số lõi xử lý và dễ dàng mở rộng cho các hệ thống đa lõi có quy mô lớn hơn. Đồng thời,

Thuật toán 3 Iterative Matrix Multiplication

Đầu vào: hai ma trận A và B kích thước $p \times p$

Đầu ra: ma trận C là kết quả với kích thước phù hợp

1: Chia ma trận A kích thước $p \times p$ thành:

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,(p-1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{\frac{p}{4},0} & a_{\frac{p}{4},1} & \cdots & a_{\frac{p}{4},(p-1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{\frac{2p}{4},0} & a_{\frac{2p}{4},1} & \cdots & a_{\frac{2p}{4},(p-1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{\frac{3p}{4},0} & a_{\frac{3p}{4},1} & \cdots & a_{\frac{3p}{4},(p-1)} \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix} = \begin{bmatrix} A_0 \\ A_1 \\ A_2 \\ A_3 \end{bmatrix}$$

2: Chia ma trận C kích thước $p \times p$ thành:

$$C = \begin{bmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,(p-1)} \\ \vdots & \vdots & \ddots & \vdots \\ c_{\frac{p}{4},0} & c_{\frac{p}{4},1} & \cdots & c_{\frac{p}{4},(p-1)} \\ \vdots & \vdots & \ddots & \vdots \\ c_{\frac{2p}{4},0} & c_{\frac{2p}{4},1} & \cdots & c_{\frac{2p}{4},(p-1)} \\ \vdots & \vdots & \ddots & \vdots \\ c_{\frac{3p}{4},0} & c_{\frac{3p}{4},1} & \cdots & c_{\frac{3p}{4},(p-1)} \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix} = \begin{bmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{bmatrix}$$

3: Thực hiện nhân ma trận song song:

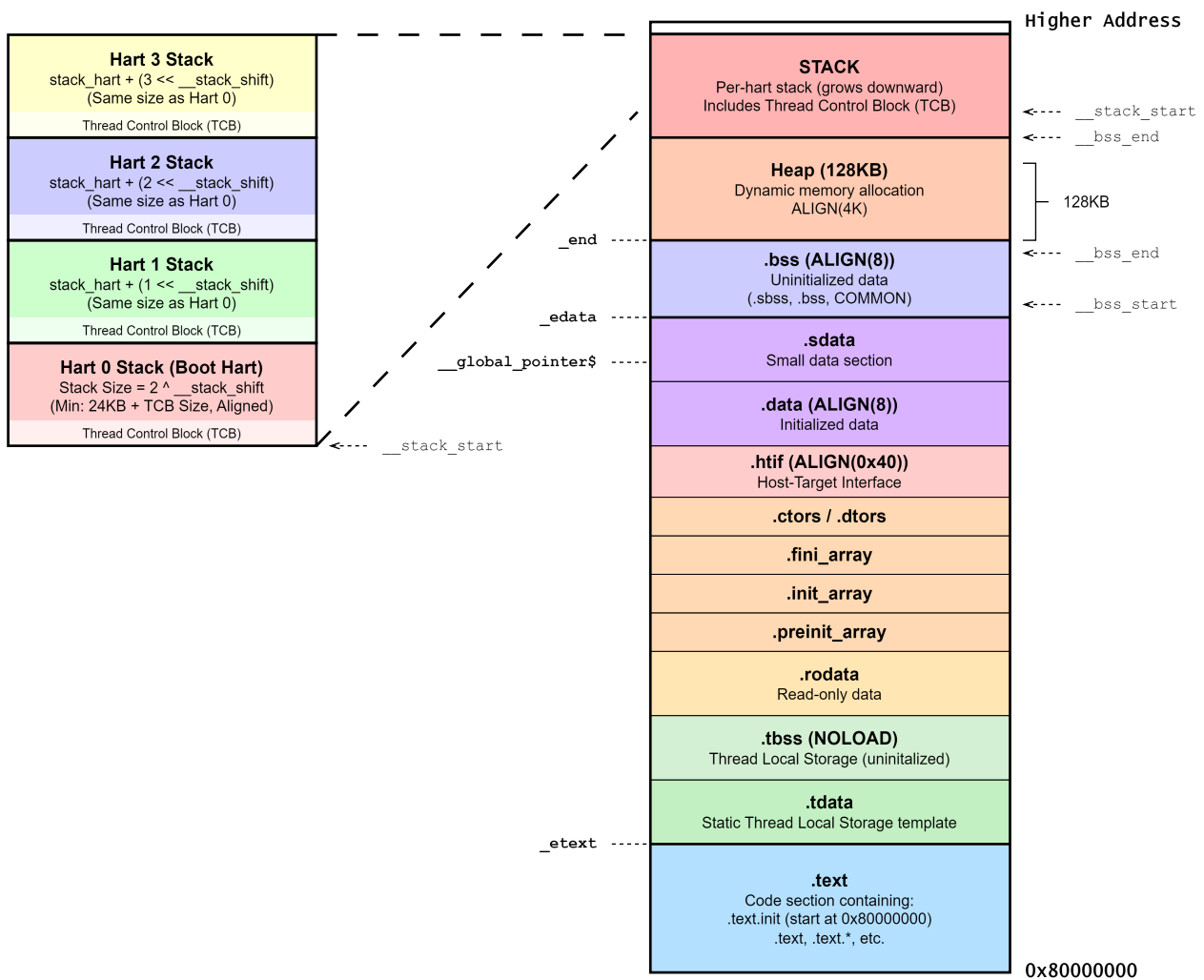
- Hart 0: $MatMul(C_0, A_0, B)$
 - Hart 1: $MatMul(C_1, A_1, B)$
 - Hart 2: $MatMul(C_2, A_2, B)$
 - Hart 3: $MatMul(C_3, A_3, B)$
-

phương pháp này tránh được tình trạng tranh chấp dữ liệu (race condition) vì mỗi hart chỉ ghi dữ liệu vào phần riêng biệt của ma trận C , trong khi ma trận A và B chỉ được đọc. Nhờ đó, các hart không gặp xung đột khi đọc/ghi và hệ thống đảm bảo an toàn dữ liệu trong quá trình thực thi song song.

3.8 Sơ đồ phân bố bộ nhớ

Trong hệ thống SoC đa lõi sử dụng kiến trúc RISC-V, bộ nhớ cần được phân bố hợp lý để bảo đảm khả năng khởi động, quản lý luồng (thread), tối ưu hiệu năng truy cập, và phân chia tài nguyên giữa các hart. Hình 3.16 thể hiện sơ đồ phân bố bộ nhớ (memory layout) từ địa chỉ $0x80000000$, là địa chỉ khởi đầu cho việc nạp mã máy.

- *.text*: Chứa mã lệnh thực thi của chương trình. Đây là vùng chỉ đọc và được nạp tại địa chỉ $0x80000000$.
- *.tdata* và *.tbss*: Dành cho dữ liệu cục bộ của từng luồng (Thread Local Storage - TLS). Vùng *.tdata* lưu các biến TLS đã khởi tạo, trong khi *.tbss* lưu các biến chưa khởi tạo, được khai báo với thuộc tính `NOLOAD`.
- *.rodata* và các mảng khởi tạo: Bao gồm *.rodata* (dữ liệu chỉ đọc), cùng các vùng *.preinit_array*, *.init_array*, *.fini_array*, *.ctors* và *.dtors*, phục vụ cho việc gọi hàm khởi tạo và hủy khởi tạo tự động.
- *.htif*: Phân vùng này dùng để hỗ trợ các lệnh gọi hệ thống thông qua giao thức Berkeley Host-Target Interface (HTIF).
- *.data* và *.sdata*: Dành cho các biến toàn cục đã khởi tạo (*.data*) và dữ liệu kích thước nhỏ được tối ưu để truy cập nhanh (*.sdata*).
- *.bss*: Chứa các biến toàn cục chưa được khởi tạo.
- *Heap*: Vùng cấp phát bộ nhớ động, thường dùng cho các hàm như `malloc()`. Trong sơ đồ này, heap có kích thước cố định là 128 KB, được căn lề theo 4 KB.
- *Stack*: Mỗi hart được cấp một vùng stack riêng biệt theo thứ tự tăng địa chỉ. Mỗi stack bao gồm một Thread Control Block (TCB) để phục vụ quản lý luồng. Stack của hart 0 (Boot Hart) bắt đầu tại `__stack_start` với kích thước tối thiểu là $2 \times \text{__stack_shift}$ (tối thiểu 24 KB + kích thước TCB). Các stack còn lại được tính theo công thức: $\text{stack_hart} + (\text{hart_id} \ll \text{__stack_shift})$.

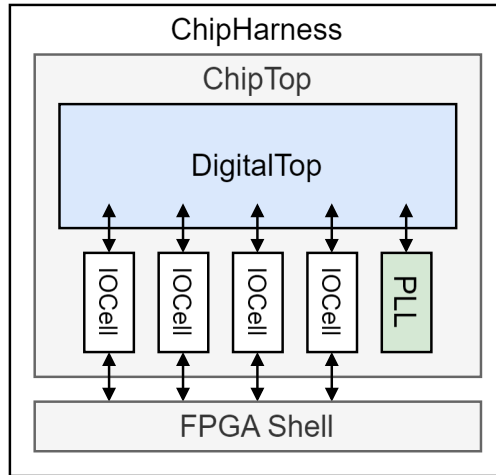


Hình 3.16: Sơ đồ phân bổ bộ nhớ trong hệ thống đa lõi RISC-V

CHƯƠNG 4: TRIỂN KHAI HỆ THỐNG TRÊN FPGA

4.1 Thiết kế và triển khai SoC trên FPGA bằng ChipYard

Phần này trình bày quy trình thiết kế và triển khai hệ thống SoC trên nền tảng FPGA bằng framework Chipyard, bao gồm cách tổ chức dự án cũng như các bước thực hiện cụ thể để đưa thiết kế phần cứng lên FPGA.



Hình 4.1: Cấu trúc tổ chức của một dự án Chipyard

Một dự án SoC trong Chipyard được tổ chức thành ba mức, như minh họa trong Hình 4.1. Ở mức *DigitalTop*, kiến trúc SoC được định nghĩa bằng ngôn ngữ Chisel. Tại đây, các module phần cứng như lõi xử lý, bộ nhớ, bus kết nối và các IP ngoại vi được tích hợp và cấu hình. Ngoài ra, không gian địa chỉ cho các thiết bị ngoại vi cũng được thiết lập.

Tiếp đến, mức *ChipTop* đảm nhiệm kết nối các chân tín hiệu I/O từ *DigitalTop* đến các *IOCell* tương ứng. Các kết nối này giúp định nghĩa giao tiếp tín hiệu I/O giữa SoC và môi trường bên ngoài. Quá trình kết nối này được gọi là IO Binders.

Cuối cùng, ở mức *ChipHarness*, các *IOCell* sẽ được ánh xạ tới các chân I/O vật lý của FPGA thông qua *FPGA Shell*. Quá trình này đảm bảo xử lý các ràng buộc thời gian (timing constraints) cũng như các yêu cầu vật lý khác để đảm bảo hệ thống hoạt động ổn định. Bước kết nối này được gọi là Harness Binders.

Dựa theo cách tổ chức trên, quy trình triển khai một SoC tùy chỉnh trên FPGA bằng Chipyard sẽ gồm các bước sau:

Bước 1: Cấu hình và tích hợp *DigitalTop*.

Bước 2: Thiết lập sơ đồ ánh xạ địa chỉ cho các thiết bị ngoại vi trong *DigitalTop*.

Bước 3: Kết nối *DigitalTop* với các *IOCell* (IO Binders).

Bước 4: Kết nối các *IOCell* với *FPGA Shell*, trong đó *FPGA Shell* bao gồm các chân I/O của *ChipHarness*.

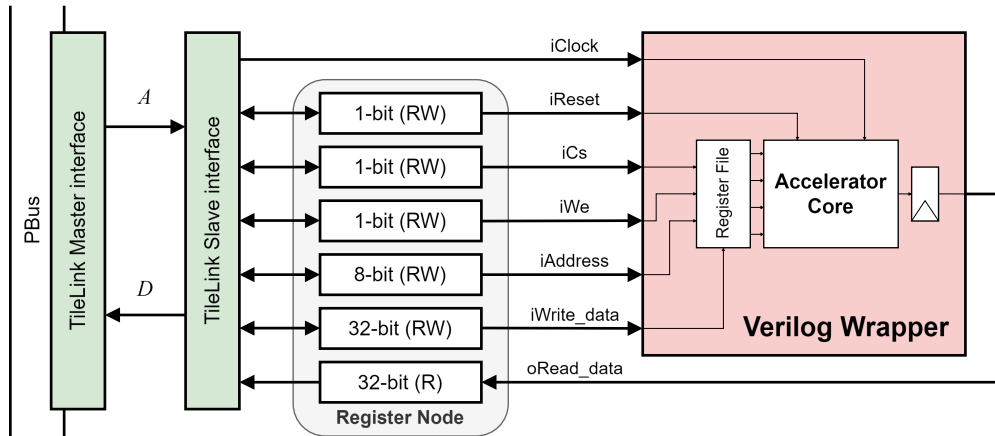
Bước 5: Xác định tần số clock cho toàn bộ hệ thống.

Bước 6: Chỉnh sửa file *Makefile* để cập nhật cấu hình mới cho quá trình tổng hợp và triển khai trên FPGA.

Các bước trên giúp đảm bảo hệ thống SoC được thiết kế có thể hoạt động chính xác trên nền tảng FPGA, từ khâu xây dựng kiến trúc phần cứng cho đến khâu triển khai thực tế.

4.2 Tích hợp các lõi gia tốc mật mã hóa nhẹ

Tất cả các bộ gia tốc mật mã trong hệ thống này được phát triển tuân thủ theo một kiến trúc tiêu chuẩn, được minh họa trong Hình 4.2.



Hình 4.2: Kiến trúc bộ gia tốc mật mã với giao diện TileLink

Ban đầu, lõi mật mã được thiết kế bằng ngôn ngữ mô tả phần cứng Verilog HDL. Sau đó, để chuẩn hóa giao diện giao tiếp giữa lõi gia tốc và vi xử lý, lõi này được đóng gói bên trong một lớp wrapper Verilog. Lớp này có vai trò quan trọng trong việc tạo ra giao diện truyền/nhận dữ liệu thống nhất, thông qua cơ chế đọc/ghi tương thích với các chuẩn giao tiếp bus phổ biến.

Tiếp theo, một lớp wrapper viết bằng ngôn ngữ Scala được xây dựng để kết nối với module Verilog này khi tích hợp vào hệ thống SoC. Wrapper này chịu trách nhiệm khởi tạo các thanh ghi điều khiển và trạng thái CSRs và ánh xạ các tín hiệu I/O của module Verilog sang không gian địa chỉ của hệ thống.

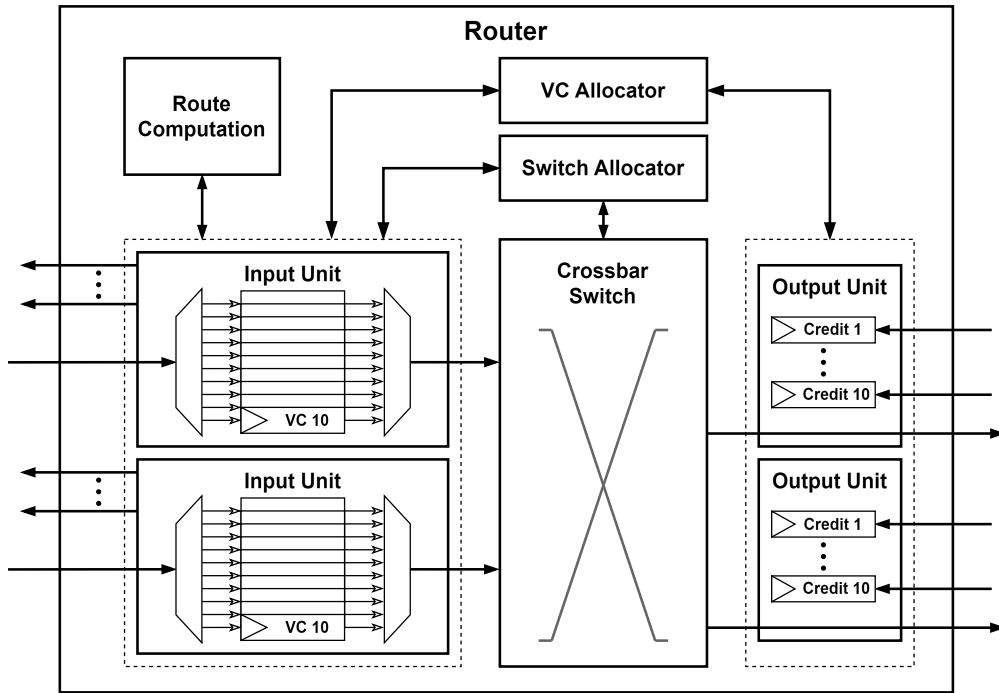
Một đối tượng *RegisterNode* [1] được tạo ra trong wrapper Scala để công bố các thanh ghi này ra phía vi xử lý. Nhờ đó, CPU có thể giao tiếp với bộ gia tốc thông

qua cơ chế truy cập bộ nhớ ánh xạ tại các địa chỉ được chỉ định. MMIO cho phép xử lý mỗi bộ gia tốc giống như một thiết bị ngoại vi, thông qua các lệnh load/store tiêu chuẩn trên các thanh ghi đã được ánh xạ. Cuối cùng, để kết nối module Scala này với bus ngoại vi của hệ thống, cặp giao diện TileLink loại TL-UL được sử dụng.

Phương pháp tích hợp này không chỉ đơn giản hóa quy trình thiết kế mà còn nâng cao tính module và khả năng tái sử dụng của hệ thống. Đặc biệt, việc tiêu chuẩn hóa wrapper Scala thành một kiểu duy nhất giúp việc phát triển và tích hợp các module gia tốc mới trở nên dễ dàng hơn, nhất là đối với những người thiết kế mới làm quen với ngôn ngữ Chisel.

4.3 Triển khai Network-on-Chip bằng Constellation

Trong khóa luận này, công cụ Constellation [34] đã được sử dụng để tự động sinh mã RTL cho kiến trúc NoC. Constellation là một công cụ mã nguồn mở, được tích hợp như một submodule bên trong Chipyard. Nhờ vậy, Constellation hỗ trợ rút ngắn thời gian thiết kế và giảm độ phức tạp trong quá trình triển khai NoC.



Hình 4.3: Kiến trúc của router

Hình 4.3 minh họa kiến trúc của một router được sinh ra bởi Constellation. Khi một packet bắt đầu truyền trong NoC, flit đầu tiên (head flit) đi vào khối *Input Unit*. Tại đây, khối *Route Computation* sẽ xử lý thông tin định tuyến để xác định ngõ ra tương ứng. Flit này sau đó được phân kênh ảo (Virtual Channel - VC) thông qua

VC Allocator, dựa trên tín hiệu phản hồi tín dụng từ bộ đệm đích. Các flit sau sẽ lần lượt đi theo cùng tuyến này. Mỗi kênh vật lý (physical channel) trong router được thiết kế với 10 kênh ảo, giúp mở rộng băng thông truyền tải dữ liệu, giảm thiểu hiện tượng tắc nghẽn đầu hàng (Head-of-Line Blocking - HOL) và tránh xảy ra các deadlock ở mức giao thức (protocol-level deadlock). Ngoài ra, mỗi kênh ảo còn được trang bị bốn bộ đệm để tối ưu hiệu suất của hệ thống khi hoạt động trong điều kiện lưu lượng cao [15].

Sau khi quá trình phân bổ kênh ảo hoàn tất, các flit sẽ cạnh tranh quyền truy cập vào khối chuyển mạch chéo (*Crossbar Switch*). Việc phân xử quyền truy cập này được quản lý bởi khối *Switch Allocator*, trong đó tích hợp bốn khối Switch Arbiters. Mỗi Switch Arbiter triển khai thuật toán phân quyền Round-Robin [34], đảm bảo việc phân chia tài nguyên một cách công bằng giữa các luồng dữ liệu.

Như đã đề cập trước đó, hệ thống sử dụng cơ chế điều khiển luồng dựa trên tín dụng. Trong cơ chế này, mỗi kênh ảo bên trong khối *Input Unit* của router đích cần gửi một tín hiệu phản hồi tín dụng về router nguồn. Tín hiệu này giúp kiểm tra trạng thái sẵn sàng của bộ đệm tại đó trước khi cho phép flit truyền qua khối crossbar. Các tín hiệu này được lưu tạm trong *Output Unit*, sau đó chuyển về *VC Allocator* để hỗ trợ quá trình phân quyền cho các flit mới.

Cách tổ chức như trên giúp đảm bảo hiệu suất truyền dữ liệu, tận dụng tài nguyên bộ đệm hiệu quả và duy trì hoạt động ổn định trong điều kiện lưu lượng cao.

4.4 Phát triển phần mềm

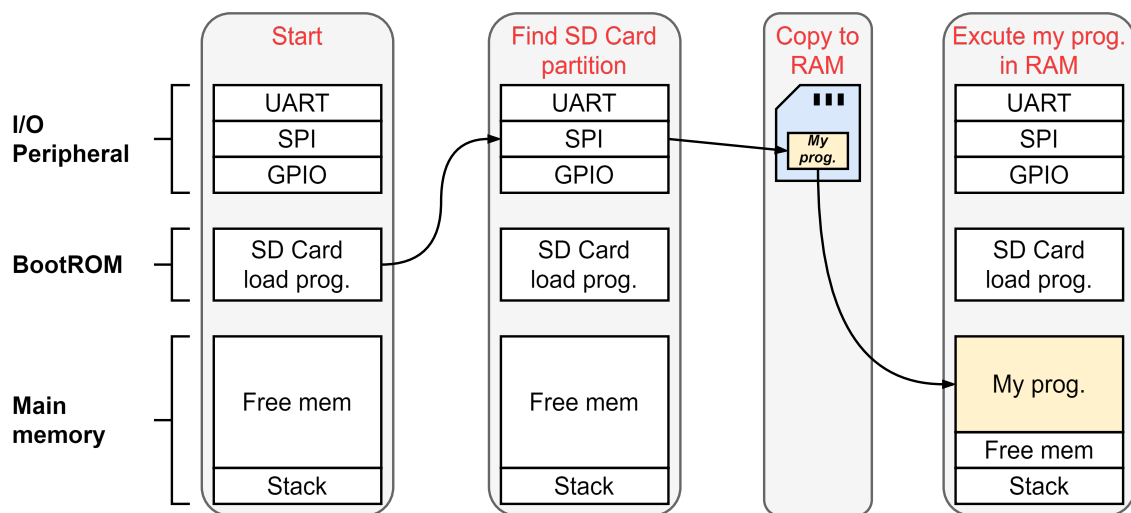
Phần này sẽ trình bày chi tiết các chương trình phần mềm được phát triển cho hệ thống SoC trong khóa luận. Các chương trình bao gồm bootloader, kiểm tra hệ thống SoC, kiểm tra các lỗi mã hóa và thực hiện thuật toán nhân ma trận.

4.4.1 Chương trình bootloader

Quy trình khởi động firmware của hệ thống được mô tả chi tiết trong Hình 4.4. Toàn bộ quá trình có thể chia thành bốn giai đoạn chính như sau:

- **Bắt đầu tại BootROM:** Sau khi hệ thống được cấp nguồn hoặc reset, vi xử lý bắt đầu thực thi lệnh từ bộ nhớ BootROM. Tại giai đoạn này, BootROM có nhiệm vụ thực hiện các thao tác cơ bản, trong đó quan trọng nhất là khởi động chương trình nạp từ thẻ SD Card. Đồng thời, các ngoại vi như UART, SPI và GPIO cũng được khởi tạo và cấu hình để sẵn sàng sử dụng.

- **Tìm phân vùng trên thẻ SD Card:** Sau khi khởi chạy, chương trình trong BootROM sẽ tìm kiếm phân vùng thích hợp trên thẻ SD Card dựa trên thông tin từ bảng phân vùng GPT (GUID Partition Table). Việc này giúp xác định vị trí lưu trữ chương trình firmware cần tải để thực thi tiếp theo.
- **Sao chép chương trình vào bộ nhớ RAM:** Khi phân vùng trên thẻ SD Card đã được xác định, chương trình sẽ được nạp từ thẻ SD Card vào bộ nhớ chính (RAM). Bước này nhằm đảm bảo chương trình sẽ được thực thi với tốc độ cao hơn so với việc chạy trực tiếp từ thẻ SD Card, đồng thời giúp CPU truy cập dữ liệu nhanh chóng và hiệu quả hơn.
- **Thực thi chương trình từ RAM:** Sau khi chương trình đã được nạp thành công vào RAM, CPU sẽ bắt đầu thực thi chương trình trực tiếp từ bộ nhớ này. Trong quá trình hoạt động, các ngoại vi như UART, SPI và GPIO vẫn duy trì khả năng giao tiếp với chương trình đang chạy. Ngoài ra, vùng nhớ còn lại trong RAM sẽ được phân bổ cho stack và không gian bộ nhớ tự do (*free memory*) để phục vụ các hoạt động của chương trình.

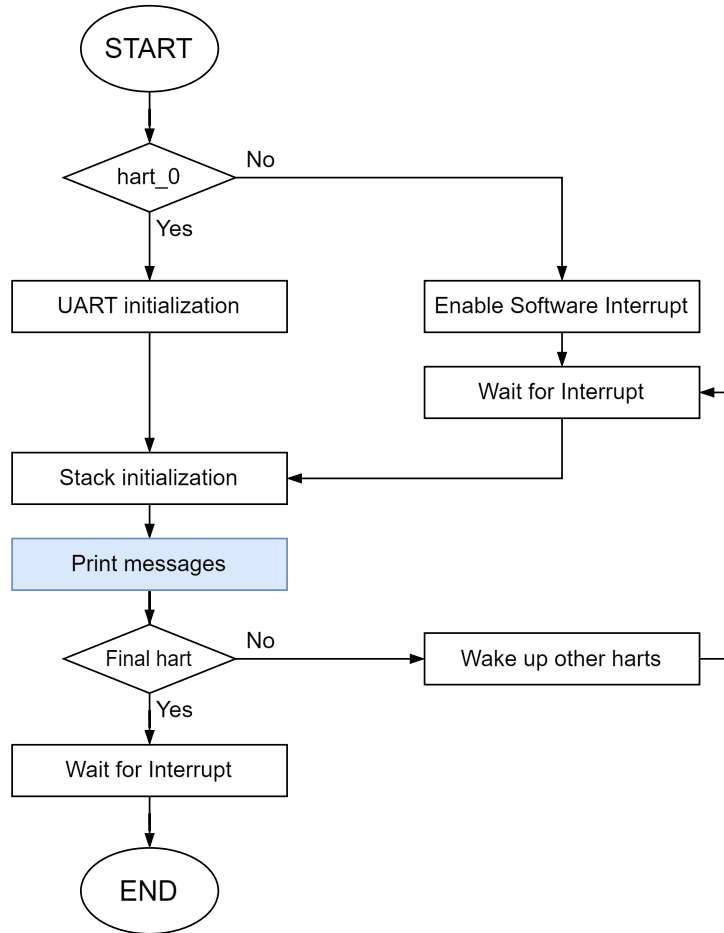


Hình 4.4: Quy trình khởi động firmware của hệ thống

Quy trình trên giúp đảm bảo quá trình khởi động hệ thống diễn ra nhanh chóng, đồng thời cho phép linh hoạt trong việc cập nhật hoặc thay đổi firmware mà không cần can thiệp vào phần cứng. Đây là phương pháp phổ biến trong các hệ thống nhúng nhằm tối ưu hóa hiệu năng và khả năng mở rộng của hệ thống.

4.4.2 Chương trình kiểm tra hệ thống SoC

Trong mục này, khóa luận xây dựng một chương trình đơn giản nhằm kiểm tra khả năng hoạt động của từng hart trong hệ thống SoC, được mô tả qua lưu đồ trong Hình 4.5.



Hình 4.5: Lưu đồ chương trình kiểm tra hệ thống SoC

Quy trình hoạt động của chương trình bắt đầu với việc tắt cả các hart khởi động đồng thời. Tuy nhiên, ngay từ đầu, chỉ có hart 0 thực hiện các tác vụ khởi tạo ban đầu, trong khi các hart còn lại sẽ tạm thời chuyển vào trạng thái chờ để tiết kiệm năng lượng. Cụ thể, các bước thực hiện như sau:

- **Khởi tạo hart 0:** Hart 0 thực hiện khởi tạo ngoại vi UART, nhằm mục đích thiết lập kết nối và cho phép in ra các thông báo kiểm tra trên terminal. Sau khi hoàn tất khởi tạo UART, hart 0 thực hiện việc cấp phát vùng nhớ stack riêng cho bản thân, kích thước 2 kB, trong vùng stack đã được phân bổ sẵn trong bộ nhớ chính. Cuối cùng, hart 0 tiến hành in thông tin xác nhận hoạt động ra terminal, nhằm xác minh quá trình khởi động thành công.

- **Trạng thái ban đầu của các hart còn lại:** Các hart từ 1 trở đi, ngay khi khởi động, sẽ tự động thiết lập bit MSIE trong thanh ghi mie của riêng từng hart. Việc kích hoạt bit MSIE cho phép các hart này sẵn sàng nhận tín hiệu ngắt phần mềm, như đã được trình bày chi tiết tại phần 3.4.2. Sau khi cấu hình, các hart này sẽ chuyển vào trạng thái ngủ và chỉ được đánh thức khi có tín hiệu ngắt, nhằm tiết kiệm tiêu thụ năng lượng cho hệ thống.
- **Đánh thức tuần tự các hart:** Sau khi hoàn tất quá trình in thông tin, hart 0 sẽ gửi tín hiệu ngắt phần mềm để đánh thức hart 1. Khi hart 1 được đánh thức, nó cũng tiến hành cấp phát vùng stack riêng cho mình, thực hiện in thông tin ra terminal, sau đó tiếp tục gửi ngắt để đánh thức hart 2. Quy trình này lặp lại tương tự đối với hart 2 và hart 3. Mỗi hart khi được đánh thức sẽ thực hiện nhiệm vụ của nó một cách độc lập: khởi tạo stack, in thông báo ra terminal và đánh thức hart kế tiếp (nếu còn).
- **Kết thúc chương trình:** Đối với hart cuối cùng (hart 3), sau khi thực hiện việc in thông tin, hart này sẽ không gửi tín hiệu đánh thức thêm bất kỳ hart nào khác. Thay vào đó, nó sẽ đi vào trạng thái chờ và chương trình kết thúc.

Quy trình trên không chỉ giúp kiểm tra khả năng hoạt động độc lập của từng hart trong hệ thống SoC, mà còn xác nhận tính đúng đắn của cơ chế giao tiếp thông qua ngắt phần mềm và khả năng phân phối vùng stack riêng biệt cho từng lõi xử lý. Đây là một bước quan trọng để đảm bảo tính ổn định và đồng bộ của các thành phần phần cứng cũng như phần mềm trong hệ thống SoC được thiết kế.

4.4.3 Chương trình kiểm tra các lỗi mật mã hóa

Phần này trình bày cách thức xây dựng chương trình phần mềm để kiểm tra hoạt động của các lõi gia tốc mật mã hóa nhẹ tích hợp trong hệ thống SoC. Mỗi lõi mật mã được ánh xạ vào một không gian địa chỉ riêng trong bộ nhớ, cho phép CPU giao tiếp với các lõi này thông qua cơ chế MMIO.

Bảng 4.1: Địa chỉ truy cập tới các lõi mật mã hóa nhẹ

Tên lõi gia tốc	Địa chỉ cơ sở
KLEIN-64	0x10006000
ChaCha	0x10007000
SHA3-512	0x10008000

Bảng 4.1 liệt kê các địa chỉ cơ sở (base addresses) của từng khối gia tốc tích hợp trong hệ thống, bao gồm các khối KLEIN-64, ChaCha và SHA3-512. Các địa chỉ này được sử dụng làm cơ sở để tính toán các thanh ghi bên trong từng lõi gia tốc, giúp CPU xác định vị trí cần đọc hoặc ghi dữ liệu trong quá trình vận hành.

Để vận hành từng lõi gia tốc, chương trình cần thực hiện các thao tác đọc và ghi lên các thanh ghi đã được ánh xạ trong không gian địa chỉ của hệ thống. Mỗi lõi mật mã cung cấp các thanh ghi chức năng với các offset cụ thể, được mô tả chi tiết trong Bảng 4.2.

Bảng 4.2: Ánh xạ thanh ghi cho các tín hiệu của bộ gia tốc mật mã

Tên thanh ghi	#Byte Offset ¹	4	3	2	1
Control	0x0000	reserved	iReset	iWe	iCs
Address	0x0004	reserved			iAddress
Write_Data	0x0008	iWrite_data			
Read_Data	0x000C	iRead_data			

¹Địa chỉ offset tương ứng với địa chỉ của các lõi mã hóa.

Cụ thể:

- Offset 0x0000, tồn tại thanh ghi *Control*, trong đó gồm ba tín hiệu điều khiển chính:
 - *iReset* (byte 3): Dừng để khởi tạo lại lõi gia tốc về trạng thái ban đầu.
 - *iWe* (byte 2): Tín hiệu cho phép ghi dữ liệu vào lõi gia tốc.
 - *iCs* (byte 1): Tín hiệu chọn lõi gia tốc để kích hoạt giao tiếp.
- Offset 0x0004 chứa thanh ghi *Address*, trong đó trường *iAddress* (byte 0) dùng để chỉ định địa chỉ thanh ghi bên trong lõi gia tốc (ví dụ như chọn thanh ghi lưu trữ key, plaintext hoặc các thanh ghi trung gian khác) mà dữ liệu sẽ được ghi hoặc đọc.
- Offset 0x0008 ánh xạ tới thanh ghi *Write_Data*. Dữ liệu đầu vào cho lõi gia tốc (chẳng hạn như plaintext hoặc key) sẽ được ghi vào thanh ghi này.
- Offset 0x000C ánh xạ tới thanh ghi *Read_Data*. Sau khi quá trình tính toán hoàn thành, dữ liệu kết quả (chẳng hạn ciphertext hoặc giá trị hash) sẽ được đọc từ thanh ghi này về CPU.

Quy trình vận hành cơ bản của chương trình kiểm tra như sau:

1. Ghi các tín hiệu điều khiển cần thiết vào thanh ghi *Control* để khởi tạo và kích hoạt lõi gia tốc.
2. Thiết lập địa chỉ thanh ghi đích bên trong lõi gia tốc bằng cách ghi giá trị tương ứng vào thanh ghi *Address*.
3. Ghi dữ liệu cần xử lý (ví dụ plaintext, key) vào thanh ghi *Write_Data*.
4. Đọc kết quả từ thanh ghi *Read_Data* sau khi lõi gia tốc hoàn thành tính toán.

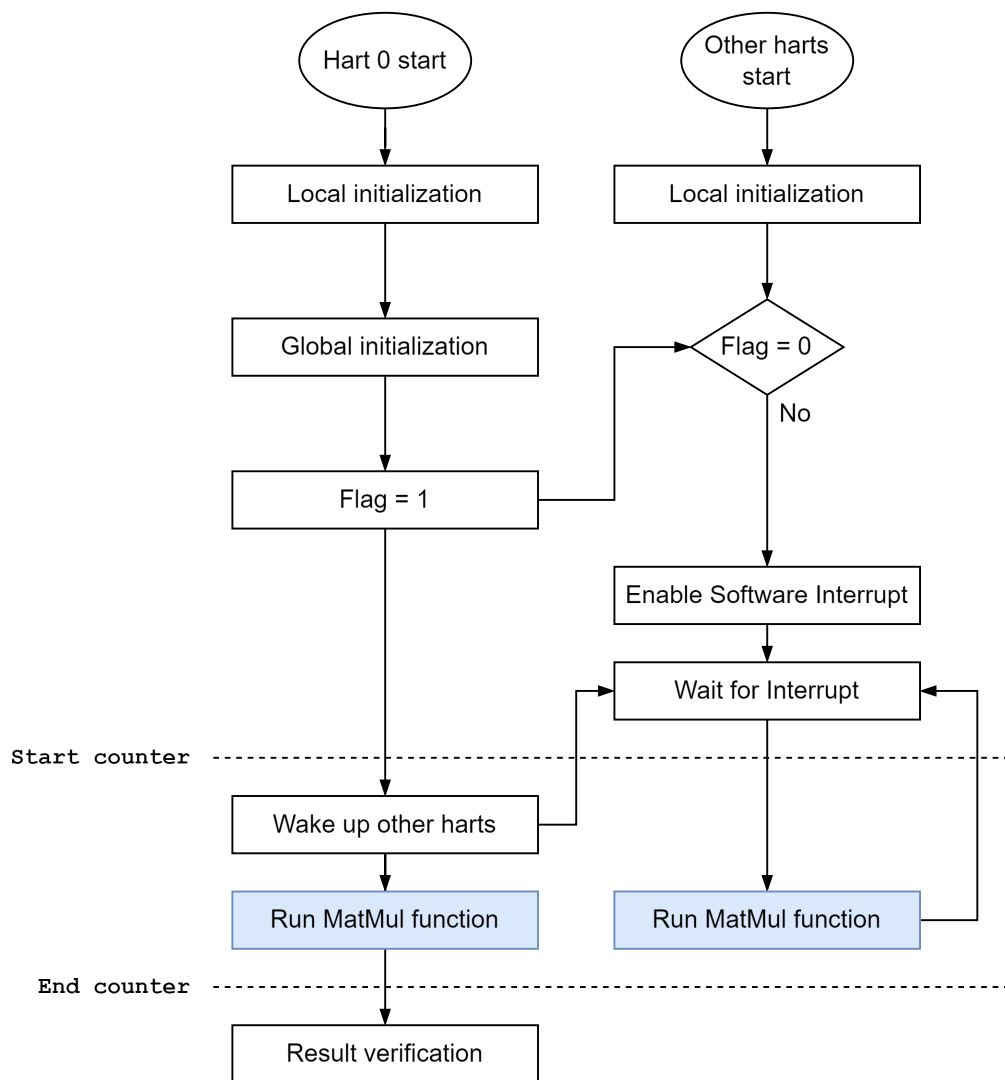
Ảnh xạ thanh ghi được thiết kế đồng nhất cho tất cả các khối gia tốc mật mã, giúp đơn giản hóa việc xây dựng phần mềm điều khiển cũng như tạo điều kiện thuận lợi cho việc tích hợp và phát triển thêm các lõi gia tốc mới trong tương lai.

4.4.4 Chương trình thuật toán nhân ma trận

Chương trình nhân ma trận này sẽ được khởi động khác với các chương trình trước một xíu như Hình 4.6. Ban đầu, tất cả các hart đều được khởi động và hoạt động ngay từ đầu. Các hart thực hiện việc khởi tạo nội bộ các thanh ghi và con trỏ liên quan tới luồng, stack và trap vector. Cụ thể, tất cả các thanh ghi được khởi tạo về 0, các thanh ghi máy trạng thái được cấu hình, đơn vị dấu động cũng được khởi tạo. Sau bước này sẽ có một biến *flag* được khởi tạo trong bộ nhớ chính. Biến này được sử dụng với mục đích làm rào chắn ngăn cản các hart khác ngoài hart 0 thực hiện việc khởi tạo toàn cục. Khởi tạo toàn cục ở đây liên quan đến việc khởi tạo cũng như cấu hình bộ nhớ chung giữa các lõi. Điều này đảm bảo rằng sẽ không xảy ra tình trạng tranh chấp dữ liệu.

Sau khi khởi tạo toàn cục xong, hart 0 sẽ ghi lại giá trị cờ *flag* = 1, do đó các hart còn lại sẽ có thể tiếp tục thực hiện tiếp chương trình phía sau. Cụ thể là các hart còn lại sẽ kích hoạt bit *MSIE* như đã nói ở phần 4.4.2 sau đó vô chế độ ngủ để tiết kiệm năng lượng.

Chương trình nhân ma trận được bắt đầu khi hart 0 đánh thức các hart còn lại, sau đó 4 hart cùng thực hiện hàm tính toán nhân ma trận như được nói ở phần lý thuyết 3.7. Điều này thể hiện khả năng tính toán song song của hệ thống.



Hình 4.6: Lưu đồ chương trình nhân ma trận

CHƯƠNG 5: KẾT QUẢ THỬ NGHIỆM

5.1 Kết quả về chức năng

Phần này trình bày kết quả kiểm tra chức năng của hệ thống SoC tích hợp bốn lõi Rocket, các lõi mã hóa nhẹ và NoC. Hình 5.1 minh họa quá trình khởi động và cập nhật firmware của chương trình *mt-hello* trên nền tảng FPGA. Theo trình tự mô tả trong Hình 4.4, dòng đầu tiên phản ánh giai đoạn thực thi của BootROM, tương ứng với cột đầu tiên bên trái. Sáu dòng tiếp theo thể hiện quá trình cấu hình ngoại vi SPI để truy cập SD Card, như trình bày ở cột thứ hai. Kế đến, hai dòng tiếp theo cho thấy quá trình sao chép dữ liệu từ phân vùng SD Card vào RAM FPGA, tương ứng cột thứ ba. Bốn dòng cuối thể hiện việc thực thi chương trình *mt-hello* đã được nạp vào RAM từ SD Card, tương ứng với cột cuối cùng.

```
Core 0: ZERO STAGE BOOT..  
CMD0  
CMD8  
ACMD41  
CMD58  
CMD16  
CMD18  
LOADING  
FINISH  
Hello from core 0, a rocket  
Hello from core 1, a rocket  
Hello from core 2, a rocket  
Hello from core 3, a rocket
```

Hình 5.1: Kết quả chương trình kiểm tra hệ thống đơn giản

Chương trình *mt-hello* có chức năng đơn giản, chỉ in thông tin lên màn hình theo định dạng “Hello from core *i*, a *core_name*”, như đã trình bày ở Mục 4.4.2. Trong đó, *i* là chỉ số của lõi, còn *core_name* là tên lõi CPU được sử dụng (trong hệ thống này là Rocket 64-bit). Kết quả thực thi xác nhận rằng hệ thống đã được xây dựng thành công ở cả phần cứng lẫn phần mềm. Về phần cứng, các lõi trong SoC đều hoạt động ổn định, giao tiếp qua UART và nạp firmware qua SPI vận hành một cách chính xác. Về phần mềm, cửa sổ terminal hiển thị đúng thông tin mong muốn, chứng tỏ phần mềm đã được xây dựng và vận hành đúng trên nền tảng phần cứng.

Tiếp theo, phần này trình bày việc đánh giá tính đúng đắn và khả năng vận hành của các bộ gia tốc mật mã nhẹ sau khi được tích hợp vào hệ thống SoC sử dụng Chipyard. Việc kiểm tra được thực hiện thông qua chương trình firmware đã được xây dựng sẵn, như đã đề cập trong phần 4.4.3.

Hình 5.2 minh họa quá trình mã hóa và giải mã của thuật toán mã hóa khối KLEIN-64 trên một vector kiểm tra. Cụ thể, chương trình thực hiện mã hóa bản rõ (*Block*) dưới dạng hexa "0xdeadbeeff000000f" với khóa (*Key*) "0x1234567890abcdef" để tạo ra bản mã (*Cipher*) "0x5a14d67698fa7e4c". Sau đó, bản mã được giải mã ngược lại bằng cùng khóa và thu được kết quả (*Decipher*) trùng khớp với bản rõ ban đầu. Kết quả này khẳng định khối gia tốc KLEIN-64 hoạt động chính xác và ổn định khi được điều khiển thông qua firmware.

```
# KLEIN-64 - Cipher and Decipher =====
Block:    deadbeeff000000f
Key:      1234567890abcdef
Cipher:   5a14d67698fa7e4c
Decipher: deadbeeff000000f
```

Hình 5.2: Kết quả thực thi mã hóa khối KLEIN-64

Hình 5.3 trình bày kết quả kiểm thử lỗi ChaCha với hai cấu hình khác nhau:

- 8 vòng, khóa 128-bit: Kiểm tra tính đúng đắn trong cấu hình tối giản.
- 20 vòng lặp, khóa 256-bit: kiểm tra khả năng sinh keystream với mức bảo mật cao hơn, đồng thời thực hiện thử nghiệm với khối tiếp theo (next block).

```
# ChaCha - 8 rounds, 128-bit key =====
Expected result: e28a5fa4a67f8c5defed3e6fb7303486aa8427d31419a729572d777953491120b64ab8e72b8deb85cd6aea7cb6089a101824beeb08814b
Result dump:    e28a5fa4a67f8c5defed3e6fb7303486aa8427d31419a729572d777953491120b64ab8e72b8deb85cd6aea7cb6089a101824beeb08814b

# ChaCha - 20 rounds, 256-bit key =====
Expected result: 76b8e0ada0f13d90405d6ae55386bd28bdd219b8a08ded1aa836efcc8b770dc7da41597c5157488d7724e03fb8d84a376a43b8f41518a6
Result dump:    76b8e0ada0f13d90405d6ae55386bd28bdd219b8a08ded1aa836efcc8b770dc7da41597c5157488d7724e03fb8d84a376a43b8f41518a6

# ChaCha - 20 rounds, 256-bit key, next block =====
Expected result: 9f07e7be5551387a98ba977c732d080dcb0f29a048e3656912c6533e32ee7aed29b721769ce64e43d57133b074d839d531ed1f28510aff
Result dump:    9f07e7be5551387a98ba977c732d080dcb0f29a048e3656912c6533e32ee7aed29b721769ce64e43d57133b074d839d531ed1f28510aff
```

Hình 5.3: Kết quả thực thi mã hóa dòng ChaCha

Kết quả cho thấy giá trị đầu ra thực tế (*Result dump*) hoàn toàn trùng khớp với giá trị kỳ vọng (*Expected result*), chứng minh lõi gia tốc ChaCha vận hành chính xác theo đặc tả thuật toán.

Đối với lõi SHA3-512, bài kiểm thử được thực hiện bằng cách băm chuỗi văn bản quen thuộc “The quick brown fox jumps over the lazy dog”, với độ dài kết quả 512-bit.

Kết quả trong Hình 5.4 cho thấy *Digest* đầu ra hoàn toàn trùng khớp với giá trị băm kỳ vọng (*Expected*), xác thực tính toàn vẹn và độ tin cậy của lõi SHA3-512 khi hoạt động trên FPGA. Ngoài ra, một test case bổ sung được thực hiện bằng cách thêm dấu chấm “.” vào cuối chuỗi văn bản. Kết quả băm thu được hoàn toàn khác so

với trường hợp ban đầu, thể hiện rõ hiện tượng tuyết lở (avalanche effect) đặc trưng của các hàm băm mật mã.

```
# SHA3-512 =====
Message: "The quick brown fox jumps over the lazy dog."
Expected: 01dedd5de4ef14642445ba5f5b97c15e47b9ad931326e4b0727cd94cefc44fff23f07bf543139939b49128caf436dc1bdee54fcb24023a08d9400
Digest: 01dedd5de4ef14642445ba5f5b97c15e47b9ad931326e4b0727cd94cefc44fff23f07bf543139939b49128caf436dc1bdee54fcb24023a08d9400

# SHA3-512 =====
Message: "The quick brown fox jumps over the lazy dog."
Expected: 18f4f4bd419603f95538837003d9d254c26c23765565162247483f65c50303597bc9ce4d289f21d1c2f1f458828e33dc442100331b35e7eb031b8
Digest: 18f4f4bd419603f95538837003d9d254c26c23765565162247483f65c50303597bc9ce4d289f21d1c2f1f458828e33dc442100331b35e7eb031b8
```

Hình 5.4: Kết quả thực thi hàm băm SHA3-512

5.2 Kết quả về hiệu suất

Bên cạnh việc kiểm chứng tính chính xác về chức năng, hiệu suất của các lõi gia tốc phần cứng cũng được phân tích thông qua so sánh với các triển khai phần mềm tương ứng. Ngoài ra, phần này còn trình bày phân tích hiệu năng giữa hệ thống bốn lõi và hệ thống đơn lõi.

Tốc độ xử lý của các bộ gia tốc mật mã được đánh giá thông qua việc đo số chu kỳ xung nhịp cần thiết để thực hiện các thao tác mã hóa hoặc băm và so sánh với các triển khai phần mềm tương đương chạy trên lõi Rocket trong cùng hệ thống. Đối với thuật toán mã hóa khối KLEIN-64, bộ gia tốc phần cứng hoàn tất quá trình mã hóa trong 494 chu kỳ và giải mã trong 701 chu kỳ, trong khi phiên bản phần mềm [13] cần tới 11,477 chu kỳ cho mã hóa và 14,225 chu kỳ cho giải mã. Điều này mang lại mức tăng tốc hơn 23 lần cho mã hóa và 20 lần cho giải mã.

```
# KLEIN-64 - Cryptography Accelerator =====
Executed cycles for encryption: 494
Executed cycles for decryption: 701
# KLEIN-64 - Software Implementation =====
Executed cycles for encryption: 11477
Executed cycles for decryption: 14225

# ChaCha20 - Cryptography Accelerator =====
Elapsed cycles: 426
# ChaCha20 - Software Implementation =====
Elapsed cycles: 21091

# SHA3-512 - Cryptography Accelerator =====
Elapsed cycles: 6311
# SHA3-512 - Software Implementation =====
Elapsed cycles: 200612
```

Hình 5.5: So sánh triển khai phần cứng và phần mềm của các lõi gia tốc

Đối với thuật toán mã hóa dòng ChaCha20, lõi gia tốc phần cứng chỉ mất 426 chu kỳ để hoàn thành, trong khi phiên bản phần mềm [27] phải mất tới 21,091 chu kỳ, đạt mức tăng tốc gần 50 lần. Tương tự, lõi SHA3-512 thực hiện phép băm trong 6,311 chu kỳ, nhanh hơn đáng kể so với triển khai phần mềm [20] vốn cần đến 200,612 chu kỳ, tương đương mức tăng tốc xấp xỉ 32 lần. Các kết quả này được tóm tắt trong Hình 5.5, khẳng định rõ ràng hiệu quả của việc chuyển các tác vụ mật mã sang phần cứng, giúp gia tăng hiệu suất tính toán và giải phóng tài nguyên CPU cho các tác vụ thời gian thực khác.

Kế đến, để đánh giá khả năng tính toán song song của hệ thống SoC đa lõi được đề xuất, thuật toán nhân ma trận đã được phát triển như trình bày ở phần 4.4.4 và chạy thử nghiệm trên cả cấu hình đơn lõi và bốn lõi. Số lượng lệnh đã hoàn thành cùng số chu kỳ thực thi được ghi nhận cho các kích thước ma trận khác nhau, như trình bày trong Bảng 5.1.

Bảng 5.1: Kết quả so sánh phép nhân ma trận

<i>Kích thước</i>	<i>Lệnh xong</i>	<i>Thời gian (chu kỳ)</i>		<i>Tăng tốc</i>	
		<i>Đơn lõi</i>	<i>Bốn lõi</i>	<i>Đề tài này</i>	<i>[33]</i>
8×8	21,781	39,474	12,918	3.06	-
16×16	164,469	286,986	76,066	3.77	-
32×32	1,279,669	2,215,466	562,785	3.94	1.75
64×64	10,097,973	20,601,024	5,319,524	3.87	1.27

Khi kích thước ma trận tăng, lợi thế về hiệu suất của cấu hình đa lõi trở nên rõ rệt. Cụ thể, với ma trận kích thước 8, hệ thống bốn lõi đạt tốc độ cao gấp 3.06 lần so với cấu hình đơn lõi. Mức tăng tốc tiếp tục tăng lên lần lượt 3.77 lần, 3.94 lần và 3.87 lần đối với các kích thước ma trận 16, 32 và 64. Sự tăng tốc gần tuyến tính này phản ánh khả năng phân phối khối lượng tính toán hiệu quả giữa các lõi, cùng với khả năng giao tiếp có độ trễ thấp nhờ kiến trúc NoC.

So với kết quả từ công trình trước đó trong [33], chỉ đạt mức tăng tốc 1.75 lần và 1.27 lần với ma trận kích thước 32 và 64, kiến trúc được đề xuất trong luận văn này cho thấy khả năng tăng tốc vượt trội. Các kết quả này khẳng định khả năng mở rộng của hệ thống và lợi ích rõ rệt của xử lý song song đối với các bài toán có yêu cầu tính toán cao.

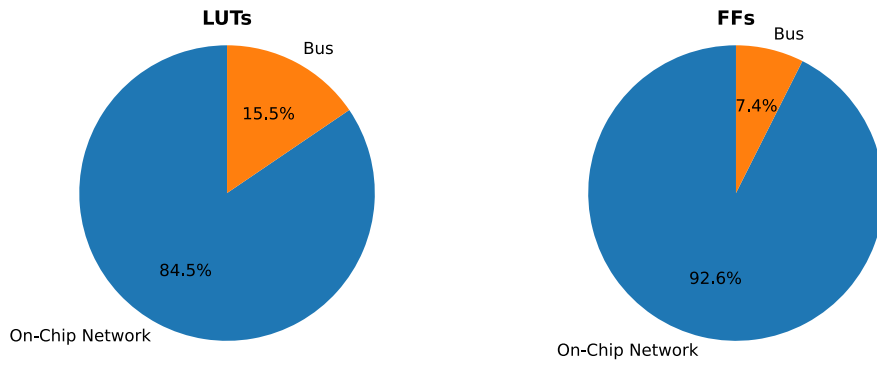
5.3 Kết quả về tài nguyên và công suất

Kết quả tổng hợp khi triển khai trên board FPGA Xilinx VC707 đã sử dụng **147,996 LUT**, **83,858 FF**, **192 BRAM** và **96 DSP**. Bảng 5.2 trình bày đánh giá chi tiết về mức sử dụng tài nguyên cho từng thành phần trong hệ thống. Các quan sát đáng chú ý bao gồm: Rocket Tile, hệ thống liên kết và các lõi gia tốc mật mã. Mỗi Rocket Tile, bao gồm CPU, FPU và bộ nhớ đệm chỉ thị/dữ liệu L1, tiêu tốn khoảng 25,035 LUT, 12,536 FF, 48 BRAM và 24 khối DSP. Do đó, với bốn tile trong hệ thống, tổng tài nguyên liên quan đến lõi tăng gần gấp bốn lần những giá trị này. Hệ thống liên kết sử dụng tổng cộng 21,126 LUT và 12,047 FF. Riêng kết nối NoC sử dụng nhiều hơn 5 lần và 12 lần tài nguyên LUT và FF so với các kết nối kiểu bus tiêu chuẩn (MBus và PBus), như được minh họa trong Hình 5.6. Mức chiếm dụng này cho thấy sự đánh đổi giữa diện tích, khả năng mở rộng và giao tiếp độ trễ thấp. Bằng cách sử dụng cấu trúc topology vòng và thuật toán định tuyến tối thiểu, các router NoC được tạo ra thông qua công cụ Constellation với mức chiếm dụng diện tích thấp, từ đó duy trì được hiệu quả thiết kế.

Bảng 5.2: Mức sử dụng tài nguyên của hệ thống

<i>Tài nguyên</i>	<i>LUT</i>	<i>FF</i>	<i>BRAM</i>	<i>DSP</i>
Multicore System	131,518	68,733	192	96
└Rocket Tile (1 Tile)	25,035	12,536	48	24
└┐CPU	5,211	2,011	0	13
└┐FPU	12,234	3,760	0	11
└┐\$I Cache	4,048	3,919	12	0
└┐\$D Cache	2,753	2,280	36	0
└┐Khác	789	566	0	0
└Interconnect System	21,126	12,047	0	0
└┐KLEIN	887	835	0	0
└┐ChaCha	2,825	2,005	0	0
└┐SHA3-512	3,524	2,335	0	0
└┐Khác	3,016	1,367	0	0
Memory IP	16,473	14,970	0	0
VC707Harness Shell	5	155	0	0

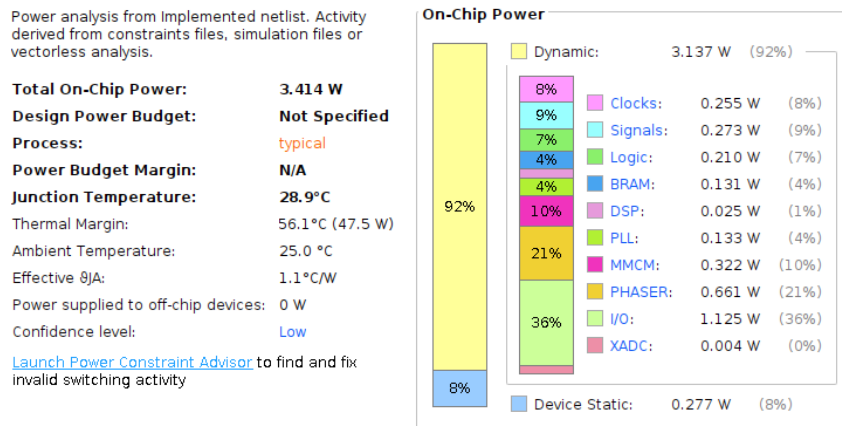
KLEIN-64 thể hiện tính hiệu quả vượt trội về sử dụng tài nguyên trong số các lõi mật mã hóa nhẹ, chỉ sử dụng 887 LUT và 835 FF, khiến nó đặc biệt phù hợp cho các thiết kế bị giới hạn nghiêm ngặt về diện tích, đồng thời cho phép phân bổ thêm



Hình 5.6: Mức sử dụng tài nguyên của hệ thống liên kết

tài nguyên cho kiến trúc đa lõi và kết nối NoC. Ngược lại, ChaCha và SHA3-512 cần nhiều tài nguyên logic hơn - lần lượt khoảng 2.8K và 3.5K LUT - phản ánh kiến trúc tính toán phức tạp hơn của chúng. Sự phức tạp cao hơn này mang lại độ chắc chắn mật mã tốt hơn, điều này có lợi cho các ứng dụng nhạy cảm về bảo mật.

Công suất tiêu thụ của hệ thống được ước lượng sau khi thực hiện tổng hợp thiết kế và ánh xạ thời gian (timing implementation) trên FPGA. Kết quả được trình bày trong Hình 5.7 cho thấy tổng công suất tiêu thụ trên chip là **3.414 W**. Trong đó, công suất tĩnh (static power) chiếm 0.277 W, tương ứng với 8% tổng công suất; còn lại là công suất động (dynamic power), chiếm 92%.



Hình 5.7: Tổng quan công suất tiêu thụ của hệ thống

Để có cái nhìn chi tiết hơn về công suất của các thành phần chức năng trong hệ thống SoC, Bảng 5.3 trình bày thống kê công suất của một số khối nổi bật. Trong đó bao gồm các lõi vi xử lý, hệ thống liên kết NoC và các bộ gia tốc mật mã.

Kết quả cho thấy rằng thành phần tiêu thụ năng lượng nhiều nhất là khối *Memory* với 2.381 W (chiếm khoảng 70% tổng công suất). Trong khi đó, toàn bộ hệ thống xử lý đa lõi chỉ tiêu thụ 0.643 W (19%), bao gồm bốn tile Rocket, mạng liên kết NoC,

Thành phần	Công suất (W)	% trên tổng
Memory IP	2.381	70
Multicore System	0.643	19
└Rocket Tile (1 Tile)	0.133	4
└Network-on-Chip	0.051	1
└SHA3-512	0.027	<1
└KLEIN-64	0.009	<1
└ChaCha	0.007	<1
└Còn lại	0.017	<1

Bảng 5.3: Thống kê về công suất theo từng thành phần

ba bộ gia tốc mật mã và các thành phần còn lại. Đáng chú ý, mỗi lõi Rocket chỉ tiêu tốn khoảng 0.133 W, trong khi NoC tiêu thụ 0.051 W. Các khối gia tốc KLEIN-64, ChaCha và SHA3-512 đều có mức tiêu thụ rất thấp, dưới 1% tổng công suất. Điều này chứng tỏ khả năng tích hợp các lõi chuyên dụng này không làm tăng đáng kể gánh nặng tiêu thụ năng lượng, phù hợp cho các ứng dụng yêu cầu tiết kiệm điện.

5.4 Tổng hợp kết quả

Bảng 5.4 trình bày tổng quan các thông số kỹ thuật chính của hệ thống SoC đa lõi được thiết kế và triển khai trong khuôn khổ đề tài. Hệ thống hoạt động ổn định ở tần số **50 MHz** - một tần số phổ biến khi triển khai trên nền tảng FPGA.

Tất cả các thành phần cấu thành hệ thống đã được trình bày chi tiết ở các mục trước. Trong đó, kiến trúc CPU sử dụng tập lệnh RV64GC với bộ nhớ đệm L1 dung lượng 16 KB cho cả I\$ và D\$. Về bộ nhớ hệ thống, ROM được cấu hình với dung lượng 64 KB, trong khi RAM có dung lượng lên đến 1 GB, đủ để thực thi các chương trình thử nghiệm phức tạp và các thuật toán mã hóa được tích hợp trong hệ thống.

Kiến trúc mạng NoC được tạo ra bằng công cụ Constellation, với thiết kế dạng vòng (*Ring*) gồm 6 router. Hệ thống sử dụng thuật toán định tuyến tối thiểu kết hợp với chiến lược chuyển mạch kiểu wormhole và cơ chế điều khiển luồng dựa trên tín dụng (*Credit-based*), giúp tối ưu độ trễ và tài nguyên phần cứng. Ngoài ra, việc tích hợp 10 kênh ảo với 4 bộ đệm còn góp phần nâng cao khả năng xử lý song song và tránh tình trạng tắc nghẽn trong mạng.

Bảng 5.4: Tổng hợp kết quả hệ thống SoC đa lõi

STT	Thông số	Cấu hình
1	Tần số hoạt động	50 MHz
2	Tổng công suất tiêu thụ	3.414 W
3	CPU —Kiến trúc tập lệnh —I\$ Cache —D\$ Cache	×4 lõi 64-bit RV64IMAFDC (RV64GC) 16 KB 16 KB
4	Uncore ¹ —Debug —CLINT —PLIC	
5	Giao tiếp ngoại vi ¹ —UART —SPI —GPIO	×1 ×1 ×1 (8-pin)
6	Bộ nhớ —ROM —RAM	64 KB 1 GB
7	Lõi mã hóa —KLEIN —ChaCha —SHA3	KLEIN64 [12] ChaCha [3] SHA3-512 [30]
8	Kiến trúc mạng NoC —Topology —Thuật toán định tuyến —Chiến lược chuyển mạch —Cơ chế điều khiển luồng —Số lượng kênh —Giao diện giao tiếp	Constellation [34] Ring (6 router) Định tuyến tối thiểu Wormhole Credit-based 10 kênh ảo (4 bộ đệm mỗi kênh) TileLink

¹Kế thừa từ cấu hình của SiFive FE310-G000 Manual [28].

CHƯƠNG 6: KẾT LUẬN

Khóa luận này đã trình bày việc thiết kế và triển khai một hệ thống SoC đa lõi dựa trên kiến trúc RISC-V, tích hợp các lõi gia tốc mật mã hóa nhẹ và kết nối NoC, trên nền tảng FPGA VC707. Hệ thống cho thấy khả năng sử dụng tài nguyên phần cứng hiệu quả, đồng thời đạt được tốc độ xử lý tác vụ mật mã cao hơn tới 50 lần so với triển khai phần mềm và tăng hiệu suất tính toán song song lên hơn 3 lần so với hệ thống đơn lõi. Kết quả này khẳng định tính hiệu quả của việc kết hợp NoC và các lõi gia tốc phần cứng trong việc xử lý bảo mật và tính toán song song, đặc biệt phù hợp với các ứng dụng điện toán tại biên.

6.1 Kết quả đạt được

Những kết quả đạt được trong khóa luận này bao gồm:

- Xây dựng thành công hệ thống SoC đa lõi với liên kết giao tiếp NoC trên FPGA.
- Tích hợp thành công các lõi mật mã hóa nhẹ KLEIN, ChaCha và hàm băm SHA3-512 vào hệ thống.
- Phát triển và thực thi thành công bootloader cùng các chương trình ứng dụng phần mềm.
- Thực hiện so sánh và đánh giá chức năng, tài nguyên phần cứng, năng lượng và hiệu suất tổng thể của hệ thống so với hệ thống hoặc cách triển khai khác.

Bên cạnh đó, các kiến thức và kỹ năng mà sinh viên đạt được sau quá trình thực hiện đề tài bao gồm:

- Hiểu biết về các chủ đề nghiên cứu đang được quan tâm hiện nay như RISC-V ISA, SoC, NoC, mã hóa nhẹ và kiến trúc đa lõi, đa luồng.
- Nắm vững quy trình phát triển hệ thống SoC dựa trên các công cụ mã nguồn mở và môi trường Linux.
- Rèn luyện kỹ năng nghiên cứu khoa học, tự giải quyết vấn đề, viết báo cáo và thuyết trình.

6.2 Hạn chế và định hướng phát triển

Tính đến thời điểm hiện tại, đề tài vẫn còn tồn tại một số hạn chế, bao gồm:

- Chưa hoàn toàn tự thiết kế và hiện thực được một CPU RISC-V hoàn chỉnh.
- Các IP tích hợp trong hệ thống phần lớn được lấy từ mã nguồn mở, chưa tự phát triển độc lập.
- Chưa thể kiểm tra một cách toàn diện các lỗi mật mã hóa nhẹ và hàm băm.
- Chưa đánh giá chi tiết kiến trúc NoC, cũng như chưa so sánh cụ thể giữa hệ thống SoC có và không sử dụng NoC.
- Chưa tối ưu được cache và quy trình trao đổi dữ liệu giữa các lõi xử lý và bộ nhớ trong hệ thống.
- Thuật toán kiểm tra khả năng xử lý song song vẫn còn hạn chế, chưa đa dạng.

Để khắc phục các hạn chế trên, đề tài đề xuất một số hướng phát triển trong tương lai gần, bao gồm:

- Tích hợp đa dạng các loại CPU RISC-V khác nhau trong cùng một hệ thống SoC không đồng nhất.
- Mở rộng số lượng lõi xử lý, phát triển kiến trúc từ multi-core lên many-core.
- Nghiên cứu các phương pháp đánh giá độc lập kiến trúc NoC, cũng như thực hiện đánh giá toàn diện hệ thống SoC có sử dụng NoC.
- Triển khai thêm private cache bên cạnh mỗi lõi xử lý theo phương pháp RoCC để giảm độ trễ xử lý của hệ thống.
- Triển khai shared coherent cache dùng chung cho các lõi RISC-V nhằm tối ưu hiệu suất và tính nhất quán dữ liệu.
- Nghiên cứu và đánh giá các giải pháp tối ưu năng lượng để phù hợp với các ứng dụng IoT.
- Thực hiện triển khai ASIC cho hệ thống.

Danh mục công trình của tác giả

1. Tôn Nữ Tâm Nhi, Trịnh Huy Hoàng, Mã Khải Minh, Lê Đức Hùng, (2024), "Thiết kế hệ thống bảo mật dữ liệu dựa trên CPU RISC-V 32-bit trên FPGA và công nghệ SKY130", *Hội nghị Quốc gia về Điện tử, Truyền thông và Công nghệ Thông tin lần thứ XXVII, REV-ECIT 2024*.

TÀI LIỆU THAM KHẢO

Tiếng Anh

- [1] Amid, Alon et al. “Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs”. In: *IEEE Micro* 40.4 (2020), pp. 10–21. DOI: 10.1109/MM.2020.2996616.
- [2] Barbirotta, Marcello et al. “Enhancing Fault Tolerance in High-Performance Computing: A Real Hardware Case Study on a RISC-V Vector Processing Unit”. In: *IEEE Open Journal of the Computer Society* 5 (2024), pp. 553–565. DOI: 10.1109/OJCS.2024.3468895.
- [3] Bernstein, Daniel J. “ChaCha, a variant of Salsa20”. In: *Workshop Record of SASC*. Vol. 8. Chicago, IL, USA: The University of Illinois at Chicago, 2008, pp. 3–5.
- [4] Cui, Enfang, Li, Tianzheng, and Wei, Qian. “RISC-V Instruction Set Architecture Extensions: A Survey”. In: *IEEE Access* 11 (2023), pp. 24696–24711. DOI: 10.1109/ACCESS.2023.3246491.
- [5] Daemen, Joan and Rijmen, Vincent. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Berlin, Heidelberg, New York: Springer Verlag, Aug. 29, 2002. ISBN: 3-540-42580-2.
- [6] Dally, William James and Towles, Brian Patrick. *Principles and Practices of Interconnection Networks*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2004. ISBN: 9780080497808.
- [7] Dong, Feihong et al. “A RISC-V Processor with Custom Instructions for Spiking Neural Network Acceleration”. In: *2024 IEEE International Conference on Integrated Circuits, Technologies and Applications (ICTA)*. 2024, pp. 222–223. DOI: 10.1109/ICTA64028.2024.10860426.
- [8] El-Hajj, Mohammed and Fadlallah, Ahmad. “Analysis of Lightweight Cryptographic Algorithms on IoT Hardware Platforms”. In: *2022 32nd International Telecommunication Networks and Applications Conference (ITNAC)*. 2022, pp. 121–126. DOI: 10.1109/ITNAC55475.2022.9998413.
- [9] Foundation, RISC-V. *Rocket Chip Generator*. 2019. URL: <https://github.com/chipsalliance/rocket-chip> (visited on 06/03/2025).

- [10] Gautschi, Michael et al. “Near-Threshold RISC-V Core With DSP Extensions for Scalable IoT Endpoint Devices”. In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25.10 (2017), pp. 2700–2713. DOI: 10.1109/TVLSI.2017.2654506.
- [11] Genc, Hasan et al. “Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration”. In: *Proceedings of the 58th Annual Design Automation Conference (DAC)*. 2021.
- [12] Gong, Zheng, Nikova, Svetla, and Law, Yee Wei. “KLEIN: A New Family of Lightweight Block Ciphers”. In: *RFID. Security and Privacy*. Ed. by Juels, Ari and Paar, Christof. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 1–18. ISBN: 978-3-642-25286-0.
- [13] GongZheng. *KLEIN_Cbenchmark*. URL: https://github.com/GongZheng/KLEIN/tree/main/KLEIN_Cbenchmark (visited on 06/03/2025).
- [14] Gonzalez, Abraham et al. “A 16mm² 106.1 GOPS/W Heterogeneous RISC-V Multi-Core Multi-Accelerator SoC in Low-Power 22nm FinFET”. In: *ESSCIRC 2021 - IEEE 47th European Solid State Circuits Conference (ESSCIRC)*. 2021, pp. 259–262. DOI: 10.1109/ESSCIRC53450.2021.9567768.
- [15] Jerger, Natalie Enright, Krishna, Tushar, and Peh, Li-Shiuan. “Router Microarchitecture”. In: *On-Chip Networks*. Cham: Springer International Publishing, 2017, pp. 75–102. ISBN: 978-3-031-01755-1. DOI: 10.1007/978-3-031-01755-1_6. URL: https://doi.org/10.1007/978-3-031-01755-1_6.
- [16] Kung, N. T. and Morris, R. “Credit-based flow control for ATM networks”. In: *Netw. Mag. of Global Internetwkg.* 9.2 (Mar. 1995), 40–48. ISSN: 0890-8044. DOI: 10.1109/65.372658. URL: <https://doi.org/10.1109/65.372658>.
- [17] Liang, Quanyi, Xie, Shaobin, and Cai, Boyang. “Intelligent Home IoT Intrusion Detection System Based on RISC-V”. In: *2023 IEEE 3rd International Conference on Power, Electronics and Computer Applications (ICPECA)*. 2023, pp. 296–300. DOI: 10.1109/ICPECA56706.2023.10076248.
- [18] Lu, Tao. “A Survey on RISC-V Security: Hardware and Architecture”. In: *CoRR* abs/2107.04175 (2021). arXiv: 2107.04175. URL: <https://arxiv.org/abs/2107.04175>.

- [19] Ma, Khai-Minh et al. “Design of an SoC Based on 32-Bit RISC-V Processor with Low-Latency Lightweight Cryptographic Cores in FPGA”. In: *Future Internet* 15.5 (2023). ISSN: 1999-5903. DOI: 10.3390/fi15050186. URL: <https://www.mdpi.com/1999-5903/15/5/186>.
- [20] mjosaarinen. *tiny_sha3*. URL: https://github.com/mjosaarinen/tiny_sha3 (visited on 06/03/2025).
- [21] Nguyen, Binh Kieu-do et al. “Hardware Software Co-Design for Multi-Threaded Computation on RISC-V-Based Multicore System”. In: *IEEE Access* 12 (2024), pp. 177312–177326. DOI: 10.1109/ACCESS.2024.3505940.
- [22] Pullini, Antonio et al. “Mr.Wolf: An Energy-Precision Scalable Parallel Ultra Low Power SoC for IoT Edge Processing”. In: *IEEE Journal of Solid-State Circuits* 54.7 (2019), pp. 1970–1981. DOI: 10.1109/JSSC.2019.2912307.
- [23] Research, UC Berkeley Architecture. *hwacha*. URL: <https://github.com/ucb-bar/hwacha> (visited on 06/03/2025).
- [24] Research, UC Berkeley Architecture. *libgloss-htif*. URL: <https://github.com/ucb-bar/libgloss-htif> (visited on 06/03/2025).
- [25] Rossi, Davide et al. “PULP: A parallel ultra low power platform for next generation IoT applications”. In: *2015 IEEE Hot Chips 27 Symposium (HCS)*. 2015, pp. 1–39. DOI: 10.1109/HOTCHIPS.2015.7477325.
- [26] Semico Research. *Semico Forecasts Strong Growth for RISC-V*. Accessed: 2025-05-26. 2019. URL: <https://www.design-reuse.com/news/7573-semico-forecasts-strong-growth-for-risc-v/>.
- [27] shiffthq. *chacha20*. URL: <https://github.com/shiffthq/chacha20> (visited on 06/03/2025).
- [28] SiFive. *SiFive FE310-G000 Manual*. Version v2p3. SiFive. 2017.
- [29] SiFive. *TileLink Specification 1.8.1*. Version 1.8.1. SiFive. 2020.
- [30] Standards, National Institute of, (NIST), Technology, and Dworkin, Morris J. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. en. 2015. DOI: <https://doi.org/10.6028/NIST.FIPS.202>. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=919061.
- [31] Tine, Blaise et al. “Accelerating Graphic Rendering on Programmable RISC-V GPUs”. In: *2022 IEEE Hot Chips 34 Symposium (HCS)*. 2022, pp. 1–15. DOI: 10.1109/HCS55958.2022.9895607.

- [32] Waterman, Andrew and Asanović, Krste. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*. RISC-V Foundation. 2017. URL: <https://www.scribd.com/document/526990741/riscv-privileged-v1-10>.
- [33] Yuan, Qiao, Pang, Wenlin, and Shi, Yuhe. “A Design of Homogeneous Multi-Core SoC with General-Purpose Floating-Point Processors”. In: *2023 IEEE 17th International Conference on Anti-counterfeiting, Security, and Identification (ASID)*. 2023, pp. 126–130. DOI: 10.1109/ASID60355.2023.10425983.
- [34] Zhao, Jerry et al. “Constellation: An Open-Source SoC-Capable NoC Generator”. In: *2022 15th IEEE/ACM International Workshop on Network on Chip Architectures (NoCArc)*. 2022, pp. 1–7. DOI: 10.1109/NoCArc57472.2022.9911299.
- [35] Zhao, Jerry et al. “SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine”. In: (2020).